

CPT104

内容完全来自于LearningMall的PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。

 GITHUB

REPOSITORY

 VISIT NOTES AUTHOR'S PAGE

CLICK ME

CPT104 **Operating Systems Concepts** 操作系统概念

■ [Week 1 Lecture]

进程 Process。

课程信息

学分：5 credits

项目	占比	其他信息
Final Exam	80%	2 小时。
CW1	20%	Week 7
CW2	20%	Week 12

模块负责人及联系方式

- 姓名: Gabriela Mogos
- 电子邮件: Gabriela.Mogos@xjtlu.edu.cn
- 办公室电话: 88161515
- 办公室地点及办公时间: SD547; 周一 13:00-14:00, 周二 15:00-16:00, 或预约

[Module Handbook] 学生须达到不低于80%的出勤率。未能遵守此要求可能导致考试失败或被禁止参加补考。

Failure to observe this requirement may lead to failure or exclusion from resit examinations or retake examinations in the following year.

内容

1. 操作系统（Operating Systems concept）的概念
2. 进程（Process Concept）概念
3. 进程调度（Process Scheduling）
4. 进程操作（Operations on Processes）
5. 进程间通信（Inter-process Communication, IPC）

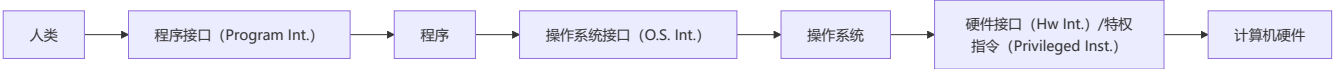
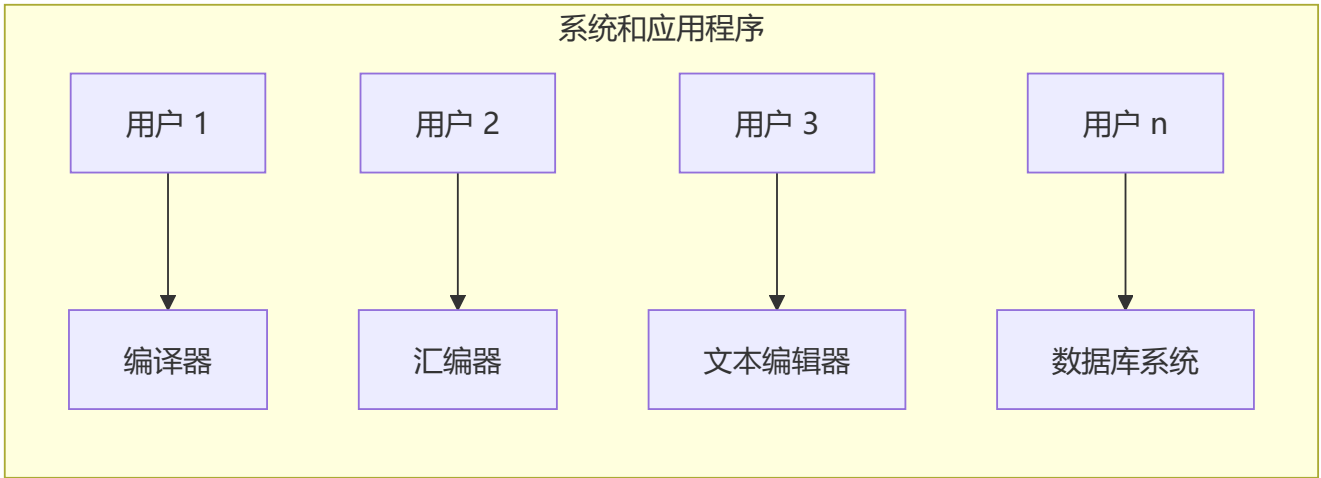
[1] 操作系统的概念

操作系统可以理解为：**用户和硬件之间的接口**。或者说，一种“**架构 (architecture)**”环境。

- 允许方便的使用；隐藏繁琐的部分
- 允许高效的使用；**并行活动 (parallel activity)**，避免**浪费的周期**
- 提供信息保护
- 为每个用户提供一个**资源片段 (a slice of the resources)**
- 作为控制程序发挥作用

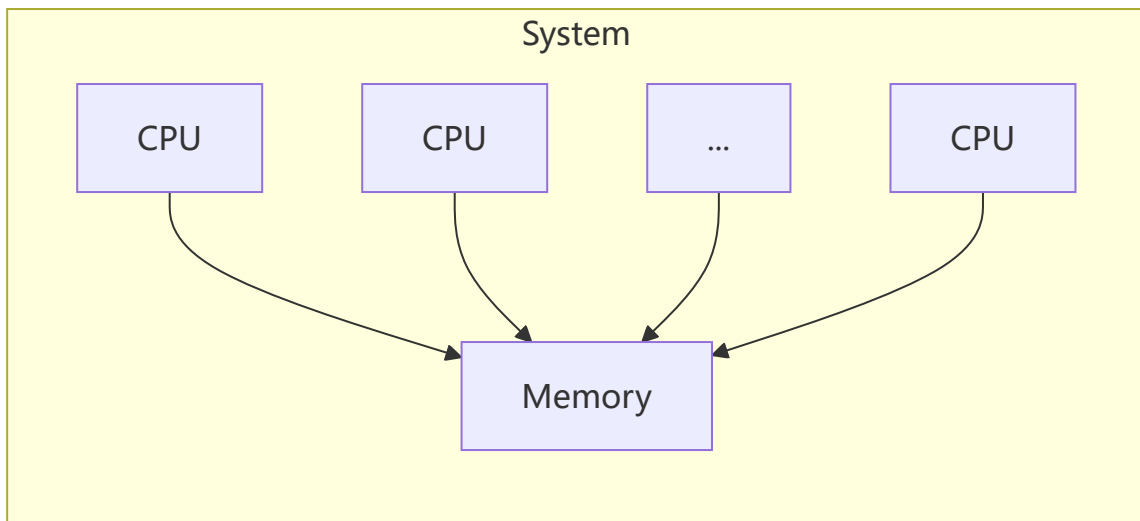
并行活动：指的是操作系统能够同时执行多个任务或进程，让多个程序或用户共享系统资源，从而提高系统的效率和响应速度。

避免浪费的周期：指的是操作系统通过合理分配和利用计算资源，避免资源闲置或浪费，确保每个资源（如 CPU、内存等）都被充分利用。

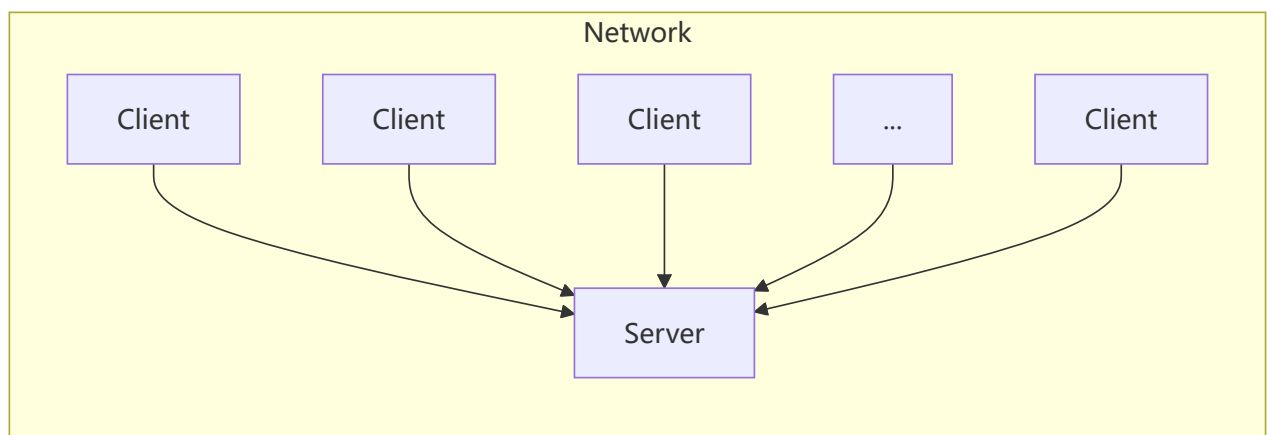


操作系统的特征：

1. **时间共享 (Time Sharing)** - 允许多个用户**同时**使用一台计算机系统。
2. **多处理 (Multiprocessing)** - 通过**共享内存**实现紧密联系的系统。通过将多个现成的处理器组合在一起，**提升计算速度**。



3. **分布式系统 (Distributed Systems)** - 通过**消息传递**实现松散联系的系统。其优点包括**资源共享**、**加快处理速度**、**提高可靠性**以及**增强通信能力**。



操作系统中最常见的操作：

1. 进程管理 (Process Management)
2. 内存管理 (Memory management)
3. 文件系统管理 (File System Management)
4. 输入/输出系统管理 (I/O System Management)
5. 保护与安全 (Protection and Security)

[2] 进程概念

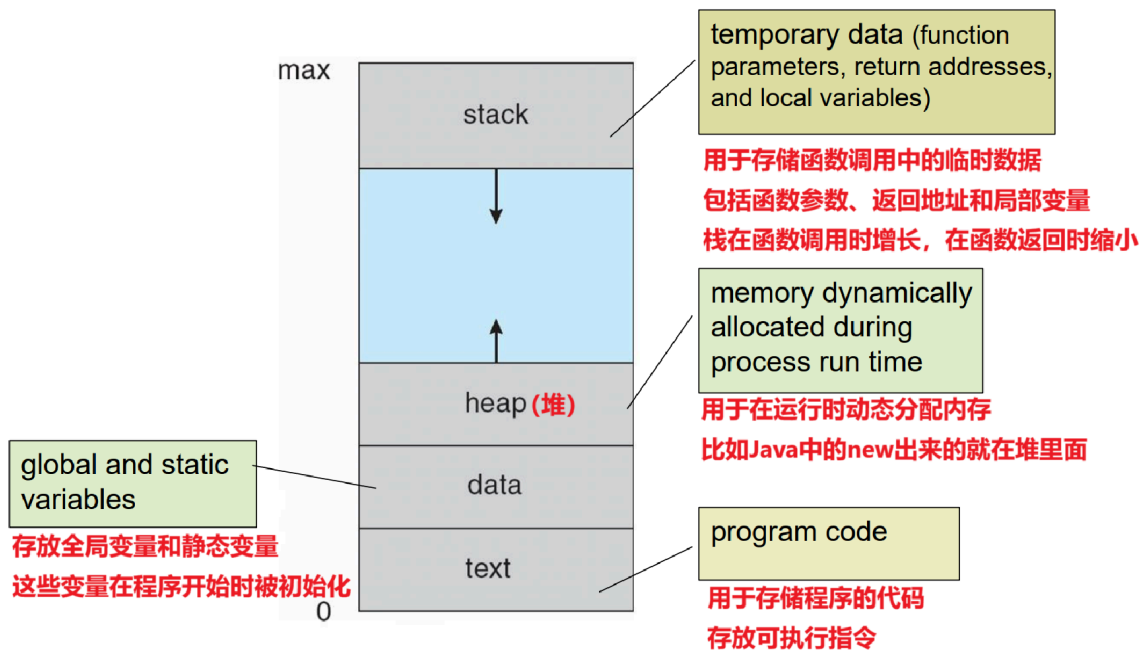
进程 = 正在执行的程序 (Process = a program in execution)

进程的执行必须**按顺序进行**(In sequential fashion)。

进程被视为“**活动 (active)**”实体。

程序被视为“**被动 (Passive)**”实体 (存储在磁盘上的可执行文件 (executable file))

程序在**可执行文件加载到内存时**成为进程。 (Program becomes process when **executable file loaded into memory**)

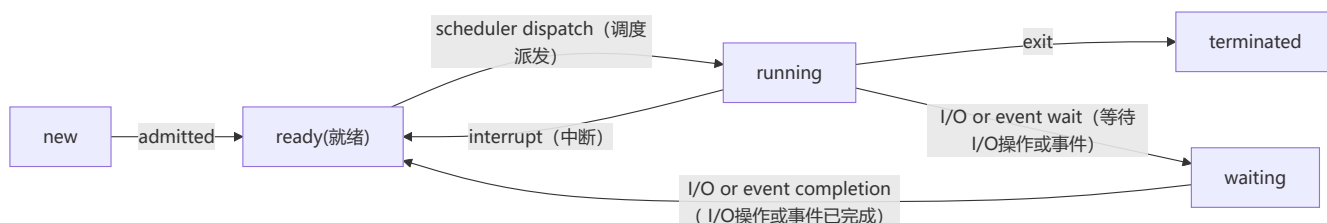


进程状态

Process Status

一个进程执行时改变**状态(status)**，进程的状态在某种程度上由该进程**当前的活动**来定义。

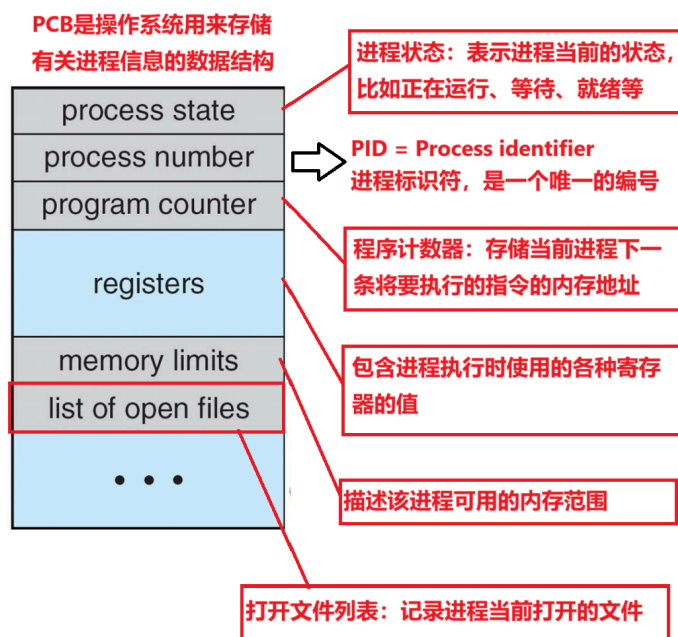
- **new (新建)**：进程正在被创建
- **running (运行)**：指令正在被执行
- **waiting (等待)**：进程正在等待某个事件发生
- **ready (就绪)**：进程正在等待被分配给处理器
- **terminated (终止)**：进程已完成执行



进程控制块 (Process Control Block, PCB) 是一种用于存储进程相关信息的数据结构。

- 每个进程都有其自身的PCB。
- PCB也被称为进程的**上下文 (context)**。
- 每个进程的PCB都存储在主内存中。
- 所有进程的PCB都存在于一个链表中。
- 在**多道程序(Multiprogramming)**设计环境中，PCB至关重要，因为它包含与同时运行的多个进程相关的信息。

多道程序：一种计算机操作系统技术，允许多个程序（程序1、程序2、程序3等）同时存在于内存中，并**轮流使用CPU**。虽然CPU同一时间只能执行一个程序，但操作系统通过快速切换，让用户感觉多个程序在同时运行。



[3] 进程调度

进程的执行由**CPU执行**和**I/O等待**的**交替序列**组成。

CPU执行：进程在运行时，CPU会执行程序中的指令

I/O等待：当进程需要进行**输入/输出**操作（如从硬盘读取文件、向显示器输出数据）时，CPU会切换到等待状态，因为这些操作需要外部设备的响应。

交替序列：进程的运行不是一直占用CPU，而是时而工作（CPU执行），时而等待（I/O等待）

三种调度器和三种队列

调度器：Process Scheduler

队列：queue

进程调度器(Process scheduler) 从内存中选择已准备好执行的进程，并将CPU分配给其中一个进程。

进程的调度队列：

- **作业队列 (Job queue)**：系统中所有进程的集合。
- **就绪队列 (Ready queue)**：所有已准备好并且等待执行的进程的集合。
- **设备队列 (Device queues)**：等待某个I/O设备（如硬盘、打印机等）的进程的集合。

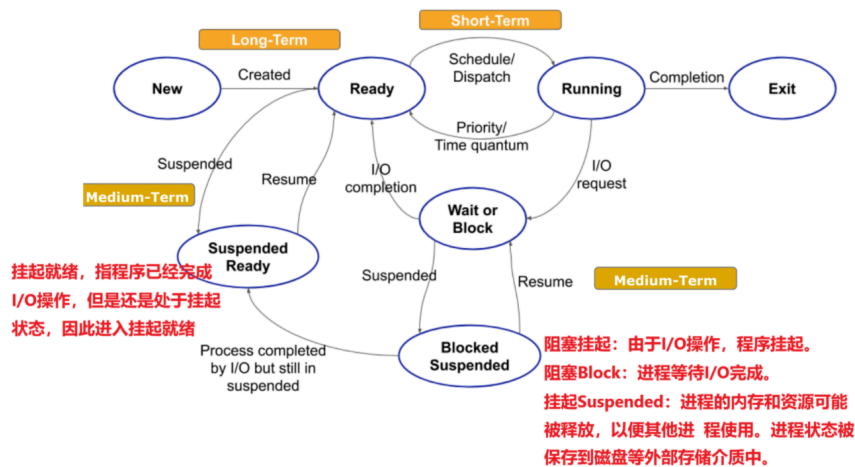
调度器种类

长期调度器(Long-Term Scheduler)：也被称为**作业调度器(Job Scheduler)**。控制系统的多道程序设计程度，管理处于**就绪 (Ready)** 状态的进程数量。

短程调度器 (Short-Term Scheduler)：也被称为**CPU调度器(CPU scheduler)**：负责从就绪队列中**选择**一个进程，并将其调度到**运行 (Running)** 状态 (CPU执行)。

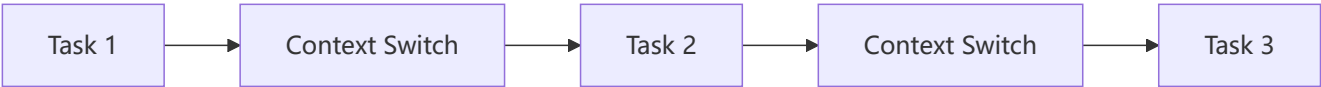
中期调度器 (Medium-term scheduler)：负责进程的内存与外存交换 (如将进程从主内存换出到磁盘，或从磁盘换入主内存)，对系统性能产生中期影响

上学期期末考有一题就是页面切换 (内存到硬盘之前切换)，算时间的，就是中期调度器做的事情。



上下文切换

当CPU切换到另一个进程时，系统必须保存旧进程的状态并通过**上下文切换 (CONTEXT SWITCH)** 加载新进程的保存状态。进程的上下文信息在**进程控制块 (PCB)** 中表示。



[4] 进程操作

操作系统必须提供必要的进程操作：**进程创建 (Process creation)**、进程终止 (Process termination)。

进程创建

父进程创建子进程，子进程又创建其他进程，形成一棵**进程树**。

资源共享有三种选项

1. 父进程和子进程共享所有资源 (类似 `Public`)
2. 子进程共享父进程资源的一个子集 (类似 `Protected`)
3. 父进程和子进程不共享资源 (类似 `Private`)

执行选项

1. 父进程和子进程**并发执行 (execute concurrently)**

2. 父进程等待直到子进程终止

进程终止

★ 进程终结的流程如下

1. 进程从所有队列中移除：

进程会从就绪队列、阻塞队列和其他相关队列中移除，表示它不再参与调度。

2. 释放进程占用的资源：

包括释放内存、关闭打开的文件、释放其他系统资源等。

3. 将终止状态返回给父进程(如果有)：

如果存在父进程，子进程会通过 `exit()` 系统调用将终止状态 (exit status) 传递给父进程。父进程可以通过 `wait()` 或 `waitpid()` 系统调用来获取这些信息。

4. 销毁或回收进程控制块 (PCB)：

当父进程获取了终止状态后，操作系统会销毁或回收子进程的 PCB。

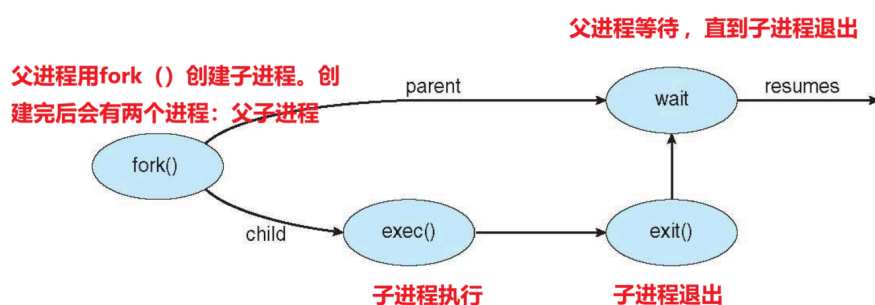
进程执行完毕最后一条语句，然后使用 `exit()` 系统调用请求操作系统删除它

子进程主动终结

父进程可以等待子进程终止执行

这是指父进程的行为。父进程可以使用诸如 `wait()` 或 `waitpid()` 的系统调用来等待其子进程的终止。在这个过程中，父进程可能会暂停执行，直到子进程完成并退出。这样做的好处是，父进程可以在子进程结束后获取子进程的退出状态信息，进行资源清理或执行其他逻辑。

- 子进程超出分配的资源



[5] 进程间通信 IPC

进程间通信 Inter-Process Communication (IPC)

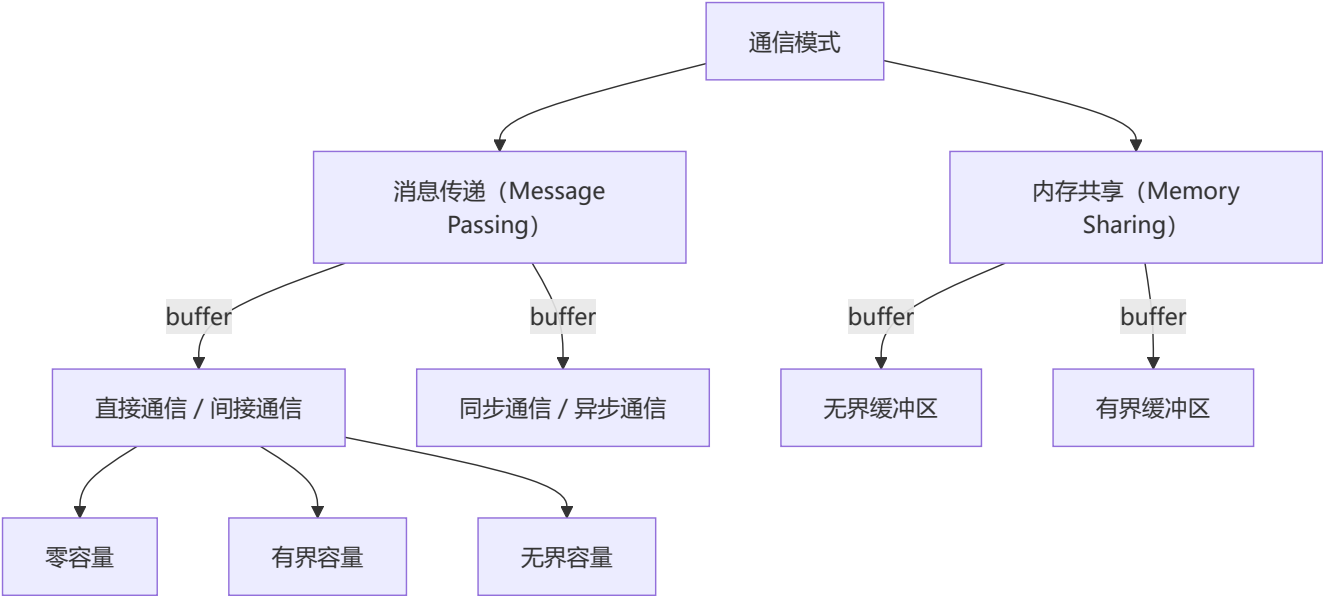
独立进程 (Independent Process) - 既不影响其他进程，也不受其他进程影响。

协作进程 (Cooperating Process) - 可以影响或被其他进程影响。

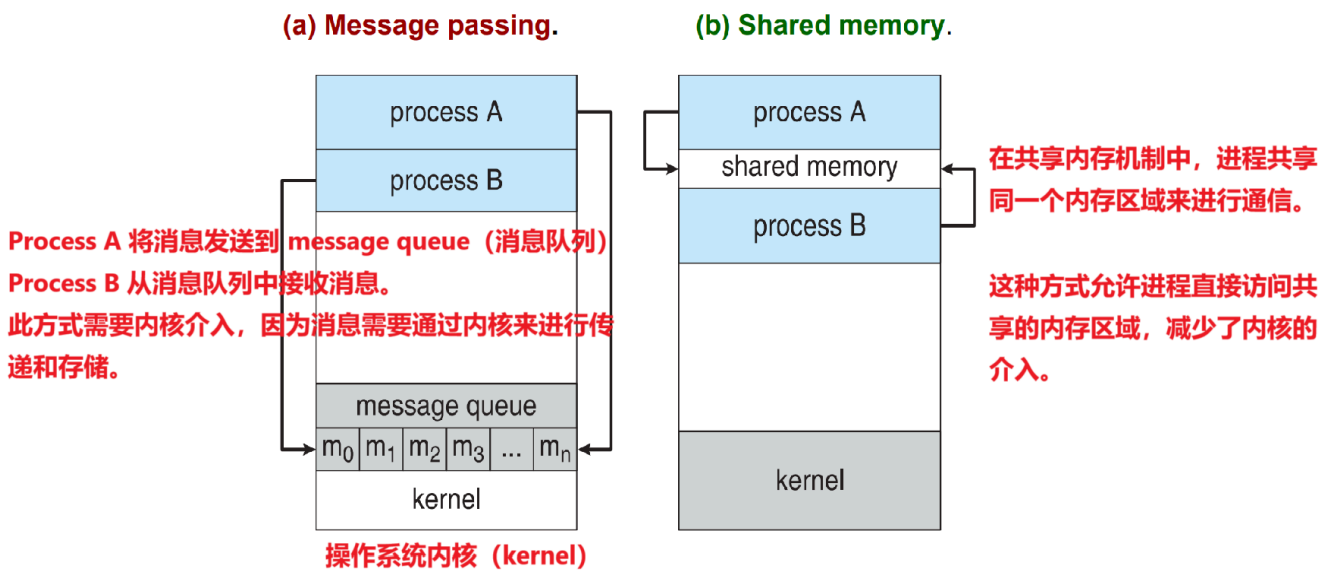
为什么需要协作进程？

- **信息共享 (Information Sharing)** - 例如，需要访问同一个文件的进程。
- **计算加速 (Computation speedup)** - 如果一个问题可以分解成同时解决的子任务，那么问题可以更快地解决。（比如java中的 `new Thread()`）
- **模块化 (Modularity)** - 将系统分解为协作的模块（例如，具有客户端-服务器架构的数据库）。
- **方便性 (Convenience)** - 即使是单个用户也可能在多任务处理，例如在不同的窗口中编辑、编译、打印和运行相同的代码。

[6] IPC 的2种通信模式



有 **Message Passing (消息传递)** 和 **Shared Memory (共享内存)** 两种。



	优点	缺点
Message Passing (消息传递)	1. 提供了严格的进程间隔离， 安全性较高 。2. 适用于分布式系统中的进程间通信。	1. 内核介入增加了通信开销，性能可能较低。2. 消息队列可能成为瓶颈，影响系统效率。
Shared Memory (共享内存)	1. 由于直接访问共享内存，通信速度快，效率高。2. 适用于需要频繁、大量数据交换的场景。	1. 共享内存需要进程间的同步机制，以防止数据竞争和一致性问题。2. 如果没有适当的访问控制和 同步机制 ，安全性较低。（例如并发操作）

综合比较

- **消息传递**适合需要强隔离和简单通信的场景，尽管其性能可能稍差。
 - **共享内存**适合需要频繁、高速数据交换的场景，但需要有额外的同步和安全考虑。
-

消息传递

[6] 通信模式中的第一种。Message-Passing

- ★ 消息传递是IPC的一种通信模式。其中消息 (Message) 可以是以下两种模式
1. **固定大小的消息(Fixed-size message)**: 消息的长度是预先定义的。
 2. **可变大小的消息(Variable-size messages)**: 消息的长度可以根据实际需要动态调整。

消息的传递至少提供如下两种操作：

1. 发送
2. 接受

如果进程 P 和 Q 需要通信，必须在他们之间建立一个**通信链路**(communication link)。

逻辑上实现链路和 `send()` / `receive()` 操作的方法有几种：

- **直接通信 / 间接通信 [Direct / Indirect communication]**
 - **同步通信 / 异步通信 [Synchronous / Asynchronous communication]**
-

直接通信 / 间接通信

[Direct / Indirect communication]

在消息传递系统中，通信进程交换的消息存储在一个**临时队列 (temporary queue)** 中。

三种缓冲机制 Buffering

1. **零容量 (Zero capacity)**：队列的最大长度为零，因此通信链路中不能有任何等待的消息。

发送方和接收方必须同时准备好，发送方发送消息后，必须等待接收方接收到消息后才能继续操作。

如果没有接收方准备好，发送方会一直阻塞，直到接收方接收消息。因此也叫做**没有缓冲空间**。
2. **有界容量 (Bounded capacity)**：队列的长度为有限值 n ，因此最多可以容纳 n 条消息。
3. **无界容量 (Unbounded capacity)**：队列的长度没有限制，理论上可以存储无限多的消息。

直接通信

- 进程必须显式地命名彼此：
 - `send(P, message)` —— 向进程 P 发送消息。
 - `receive(Q, message)` —— 从进程 Q 接收消息。
- 直接通信的实现方式是进程使用特定的**进程标识符 (specific process identifier)** 进行通信，但在某些场景下，预先确定发送方是困难的。

间接通信

- 创建一个新的**邮箱(mailbox)** (端口 port) 。
- 通过**邮箱**发送和接收消息：
 - `send(A, message)` —— 向邮箱 A 发送消息。
 - `receive(A, message)` —— 从邮箱 A 接收消息。
- 销毁邮箱。

同步通信 / 异步通信

[Synchronous / Asynchronous communication]**

消息传递可以是**阻塞 (blocking)** 或**非阻塞 (non-blocking)** 的。

阻塞 (BLOCKING) 被认为是同步的

- 阻塞发送 (Blocking send) : 发送方会被阻塞，直到消息被接收为止。
- 阻塞接收 (Blocking receive) : 接收方会被阻塞，直到有消息可用为止。

非阻塞 (NON-BLOCKING) 被认为是异步的。

- 非阻塞发送 (Non-blocking send) : 发送方发送消息后可以继续执行。
- 非阻塞接收 (Non-blocking receive) : 接收方会收到以下两种情况之一：
 - 一个有效的消息
 - 空消息 (Null message)

内存共享

[6] 通信模式中的第二种。Shared Memory

共享内存区域：由协作进程共享的一块内存区域。

信息交换：进程通过读取和写入共享区域中的所有数据来交换信息。

两种缓冲区类型：

- **无界缓冲区 (Unbounded-buffer)** : 对缓冲区的大小没有实际限制。
- **有界缓冲区 (Bounded-buffer)** : 假设缓冲区的大小是固定的。

▲ [Week 1 Tutorial]

二进制加减法

Q&A

来自LMO的quiz (不计分, Optional)

1. A Process Control Block(PCB) does not contain which of the following?

- A. Code B. Stack **C. Bootstrap program** D. Data

进程控制块 (PCB) 不包含以下哪一项?

答案: C; 进程控制块 (PCB) 是操作系统用于**管理进程的数据结构**, 通常包含以下信息:

进程状态 (Process State)、程序计数器 (Program Counter)、寄存器值 (Register Values)、栈 (Stack)、数据 (Data)、代码 (Code)、资源信息 (Resource Information)、调度信息 (Scheduling Information)。

2. A process can be terminated due to _____

- A. normal exit** B. fatal error C. killed by another process D. all of the mentioned

一个进程可能由于以下哪种原因被终止?

答案: A;

3. What is a short-term scheduler?

- A. It selects which process has to be executed next and allocates CPU**

B. It selects which process has to be brought into the ready queue

C. None of the mentioned

D. It selects which process to remove from memory by swapping

什么是短期调度?

答案: A; 参考 [3] 进程调度 -> 三种调度器和三种队列

4. **Messages** sent by a process _____

A. have to be a variable size

- B. can be fixed or variable sized**

C. have to be of a fixed size

D. None of the mentioned

答案: B; 首先这里的Message是消息的意思, 我们知道进程间通信模式IPC: **消息传递+内存共享**。因为这题提到的是**Message**, 和另一种**内存共享**无关。所以这题应该在**消息传递**中找。在 [6] IPC 的2种通信模式 -> **消息传递** 中写了, 消息既可以是固定长度, 也可以是可变长度。

5. Operating system's fundamental responsibility is to control the

A. Termination of Processes (终结进程)

- B. Execution of Processes(执行进程)**

C. Control of Processes(控制进程)

D. Access of Processes (访问进程)

操作系统的基本职责是控制_____

答案: C; 操作系统的基本职责集中在 **程序 (进程) 执行** 上

6. What is a Process Control Block (PCB) ?

A. A secondary storage section

B. Process type variable

C. Data Structure

D. A Block in memory

答案: C; PCB是操作系统中用于描述和管理进程的数据结构;

7. Essential element for a process is

A. Time

B. Code

C. Program

D. Process ID (PID)

进程的基本要素是什么?

答案: C;

进程是操作系统中的一个执行实体, 其基本要素包括:

- **进程 ID (Process ID, PID)** : 唯一标识一个进程, 是操作系统管理和调度进程的关键。
- **代码段 (Code)** : 程序的执行指令。
- **数据段 (Data)** : 程序运行时的变量和数据结构。
- **程序计数器 (Program Counter, PC)** : 指向下一条要执行的指令。
- **资源**: 分配的内存、文件句柄等。

8. If process is running currently executing, it is in running

A. Mode (模式)

B. Process (进程)

C. State (状态)

D. Program (程序)

如果进程当前正在执行, 它处于运行__。

答案: C; 进程Process 在执行时被描述为处于 **运行状态 (Running State)**。这是进程生命周期中的一个状态, 表示进程正在 CPU 上执行其指令。参考 [2] **进程概念 -> 进程状态**。

9. What will happen when a process terminates?

A. Its process control block is de-allocated (它的进程控制块被释放)

B. Its process control block is never de-allocated (它的进程控制块永远不会被释放)

C. It is removed from all, but the job queue (它从所有队列中移除, 除了作业队列)

D. It is removed from all queues (它从所有队列中移除)

当一个进程终结时，会发生什么？

答案：D；简而言之，一个进程结束会发生：队列中移除进程，释放资源，`exit()` 返回结束状态给父进程，系统销毁该进程的PCB。

A项知识其中一个。BC都不对。答案是D。

具体参考 [4] 进程操作 -> 进程终止

10. The state of a process is defined by _____

- A. the activity just executed by the process (进程刚刚执行的活动)
- B. the activity to next be executed by the process (程接下来要执行的活动)
- C. the current activity of the process (进程当前的活动)
- D. the final activity of the process (进程的最终活动)

进程的状态是_____定义的。

答案：C；进程的状态 (Process State) 是由其 **当前的活动** 决定的。操作系统会根据进程当前在执行什么操作来定义其状态。参考 [2] 进程概念 -> 进程状态

11. In indirect communication between processes P and Q _____

- A. none of the mentioned
- B. there is another machine between the two processes to help communication
- C. there is another process R to handle and pass on the messages between P and Q
- D. there is a mailbox to help communication between P and Q

在进程 P 和 Q 之间的间接通信中___。

答案：D；参考 [6] 同步通信 / 异步通信 -> 异步通信。

12. Information in the proceeding list is stored in a data structure called _____

- A. Process Access Block
- B. Data Blocks
- C. Process Control Block
- D. Process Flow Block

答案：C；进程控制块PCB。

13. Message passing system allows processes to _____

- A. communicate with one another by resorting to shared data
- B. name the recipient or sender of the message
- C. communicate with one another without resorting to shared data.
- D. share data

答案：C；消息传递（系统）允许进程：在不依赖共享数据的情况下相互通信。消息传递（系统）不依赖数据共享。

14. What is the ready state of a process?

- A. when process is scheduled to run after some execution (当进程被调度为在某些执行后运行)
- B. when process is using the CPU (当进程正在使用 CPU)

- C. when process is unable to run until some task has been completed (当进程无法运行直到某些任务完成)
- D. none of the mentioned (以上都不是)

进程的就绪状态是什么？

答案：A；B是**运行状态 (Running State)**。C是**阻塞状态 (Blocked State)**。

14. When a program is loaded into memory it is called as

- A. Procedure (过程)
- B. Register
- C. Table
- D. Process (进程)

当一个程序被加载到内存中时，它被称为

答案：A。当一个程序从存储设备（如硬盘）加载到内存中时，操作系统会为其创建一个 **进程 (Process)**。进程是程序的一个实例。

Binary Number

The number 394 is said to be written in **base 10(以10为基数)**.

$$394 = 3 \times 100 + 9 \times 10 + 4 \times 1 = 3 \times 10^2 + 9 \times 10^1 + 4 \times 10^0,$$

在数字 394 中，数字 3 的**权重(weight)**是 100，数字 9 的权重是 10，数字 4 的权重是 1。

注意：数字的权重从**右到左**递增。

二进制数以 2 为基数，仅需要数字 0 和 1。

为了区分，下文用下标代表几进制。例如：10₁₀ 代表十进制的10，用10₂代表二进制的2。

任意进制之间转换

如果想让 8 进制257₈转二进制，那么可以：

- 1. 257₈ 转 10 进制
- 2. 10 进制再转 2 进制

任意进制转十进制

例如 8 进制257₈转10进制

$$ans = 2 * 8^2 + 5 * 8^1 + 7 * 8^0 = 128 + 40 + 7 = 175$$

十进制转任意进制

例如 175₁₀ 转 2 进制

步骤	被除数 (十进制)	商	余数
1	175 ÷ 2	87	1

步骤	被除数（十进制）	商	余数
2	$87 \div 2$	43	1
3	$43 \div 2$	21	1
4	$21 \div 2$	10	1
5	$10 \div 2$	5	0
6	$5 \div 2$	2	1
7	$2 \div 2$	1	0
8	$1 \div 2$	0	1

不断进行触发求余后，把余数**倒过来写**。得到 10101111_2

小数形式二进制

将二进制数 0.1101_2 转换为 10 进制形式。

对于这种类型的二进制数，小数点后的第一位权重为 2^{-1} ，第二位为 2^{-2} ，依此类推。

二进制权重	2^{-1}	2^{-2}	2^{-3}	2^{-4}
权重值	0.5	0.25	0.125	0.0625
二进制数字	1	1	0	1

$$ans = 1 * 2^{-1} + 1 * 2^{-2} + 0 * 2^{-3} + 1 * 2^{-4} = 0.5 + 0.25 + 0.0625 = 0.8125_{10}$$

加减法

加法：

求 $101_2 + 110_2 + 1011_2$

101

110

+ 1011

1222

加完之后进行进位，满2进1

11110

减法：

求 $1101_2 - 111_2$

1101

- 111

0110

Q&A

1. 使用 n 位**非小数(non-fractional)**的二进制数，能够表示的最大（十进制）数是多少？

答案： $2^n - 1$ 。

例如，不考虑小数。使用2位二进制数，能表示 00, 01, 10, 11 四个数。其中最大的是 $11_2 = 3_{10}$ ，因此是 $2^2 - 1 = 3_{10}$ 。

2. 求 11.1101_2 的十进制

答案： 3.8125。

$$ans = (1 * 2^1 + 1 * 2^0) + (1 * 2^{-1} + 1 * 2^{-2} + 2^{-4}) = 3 + 0.5 + 0.25 + 0.0625 = 3.8125$$

■ [Week 2 Lecture]

Thread 线程

内容

线程 Thread

- 概述 / 什么是线程？
- 多核编程 (Multicore Programming)
- 多线程模型 (Multithreading Models)
- 线程库 (Thread Libraries)
- 隐式线程 (Implicit threading)
- 线程问题 / 设计多线程程序 (Threading issues / Designing multithreaded programs)

[1] 什么是线程

基本概念

回顾：

1. **CPU（中央处理器）** 是计算机中的一块硬件，用于执行二进制代码。
2. **操作系统（OS）** 是一种软件，负责**调度(schedule)**程序的 CPU 使用权。
3. **进程** 是一个正在执行的程序。

Process is a program that is being executed.

4. **线程** 是进程的一部分。

Thread is part of a process.

具体而言：

1. **执行线程(Thread of execution) 是最小的可编程指令序列**，它能够被调度器独立管理并由 CPU 独立于父进程执行。

最小的可编程指令序列: smallest sequence of programmed instructions.

[补充]：

线程是由一系列指令组成的最小执行单元。线程可以直接被OS调度给CPU执行而不依赖于进程。

2. **线程** 是一种轻量级进程，可以被调度器独立管理。

[补充]：

线程比进程更加轻量。创建和管理线程的**开销**更小。

线程与进程的主要区别在于：线程共享进程的资源（如内存空间），而进程拥有独立的资源。

3. 线程 按顺序执行一系列指令（同一时间内只能执行一件事）。

[补充]：

线程是一个**单控制流**，它按指令顺序执行代码。**在单个线程中**，一次只能执行一个操作，**不会出现并发执行的现象**。

4. **线程** 是**进程内**的控制流。

A thread is a flow of control within a process.

[补充]：

一个进程可以包含多个线程。这些**线程共享进程的资源**（如内存、文件等），但各自拥有独立的执行路径。

5. 当一个进程中有多个线程运行时，进程会提供**共享内存 (the process provides common memory)**。

6. 应用程序的多个任务可以通过独立线程实现。

Multiple tasks with the application can be implemented by separate threads.

[补充]：

通过多线程，程序可以同时处理多个任务。每个线程可以独立执行不同的操作，从而提高程序的并发性和效率。例如，在一个程序中，一个线程可以处理用户输入，另一个线程可以执行计算任务。

操作系统的视角 (O.S. view)：线程是一个独立的指令流，可以由操作系统调度运行。

软件开发者的视角 (Software developer view)：线程可以看作是一个“过程”，它**独立**于主程序运行。

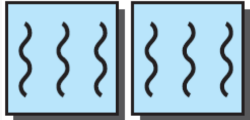
顺序程序和多线程程序

1. **顺序程序 (Sequential program)：**

程序中只有一个指令流。

2. **多线程程序 (Multi-threaded program)：**

程序中包含多个指令流（需要使用多个核心/CPU）。

	单程序	多程序
单线程		
多线程		

线代表：Instruction Trade

例如，Microsoft Word 编辑器是一个程序（存在磁盘中的静态代码）。当用户双击 Word，OS 就会创建 Word 进程。随后，

用户在 Word 编辑器中输入文本 -> 相关线程的变化：

- 打开 Word 编辑器中的文件（一个线程），
- 用户输入文本（一个线程），
- 文本自动格式化（另一个线程），
- 文本自动检查拼写错误（另一个线程），
- 文件自动保存到磁盘（另一个线程）。

线程的优点

1.响应性 (Responsiveness)

- 允许程序在部分被阻塞的时候，或者执行耗时操作的时候，继续运行。

2.资源共享与经济性 (Resource Sharing and Economy)

- 线程共享地址空间、文件、代码和数据。
- 避免资源消耗。
- 执行更快的上下文切换。

还记得上下文切换吗？意思是切换执行对象。

3.可扩展性 (Scalability)

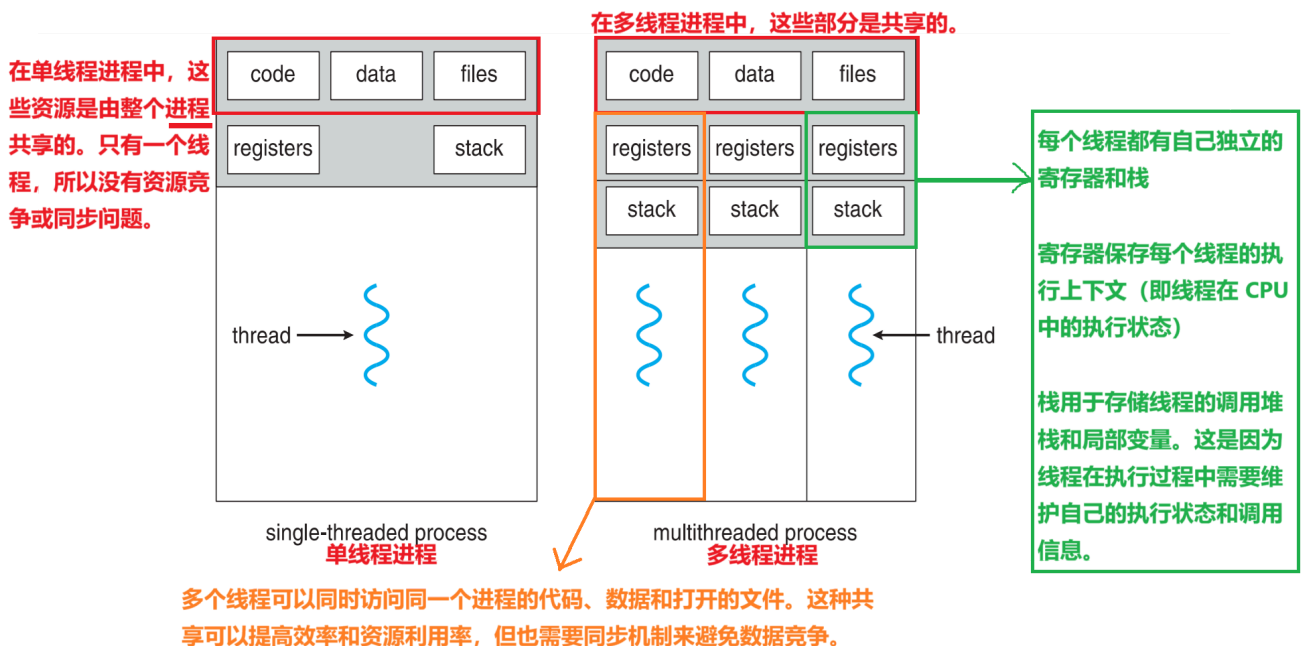
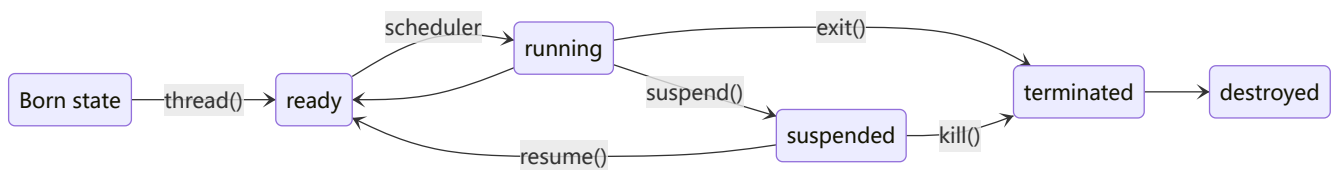
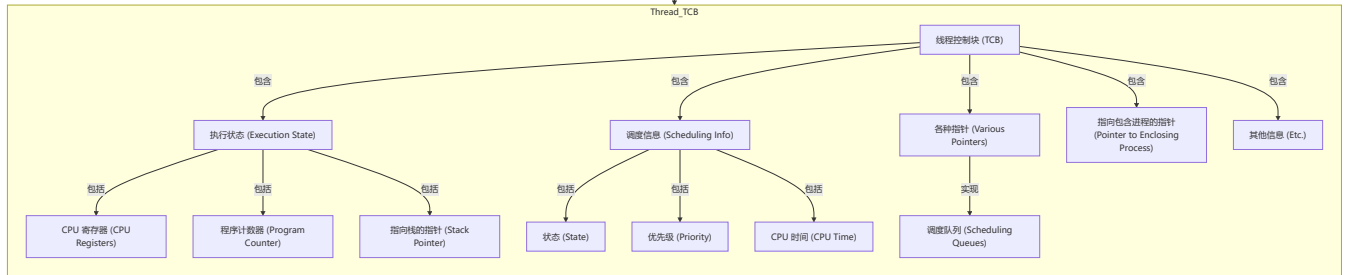
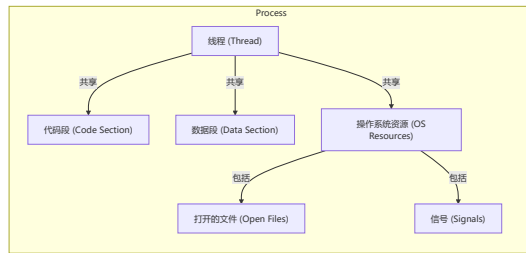
- 将代码、文件和数据放置在主内存中。
- 将线程分配到每个 CPU 上。
- 线程可以在不同的处理核心上并行运行 Running in parallel。

4.死锁避免 (Deadlock Avoidance)

线程控制块 TCB

Thread Control Block, TCB

线程控制块 (Thread Control Block, TCB) 存储了线程的相关信息。



[2] 多核编程

Multicore Programming

单核系统上的 并发执行(Concurrent execution)

并发意味着多个任务能够启动、运行，并且操作系统在它们之间快速切换 ➡ 任务是**交错执行(interleaved)**的。并发创造了**并行执行**的假象。

- 在单核系统中，同一时间只能运行一个任务，但操作系统通过快速的上下文切换（context switching）让多个任务交替执行，从而实现 **任务的交错（interleaved）**。
- 例如，假设有三个任务A、B、C，操作系统可能会先运行A一小段时间，然后切换到B，再切换到C，再回到A，如此反复。
- **并发创造了一种并行执行的错觉。**
- 虽然单核系统同一时间只能执行一个任务，但对于用户来说，多个任务似乎在同时进行（例如，你在听音乐的同时写文档）。这种错觉是通过快速的上下文切换实现的。

多核系统上的并行执行(Parallelism)

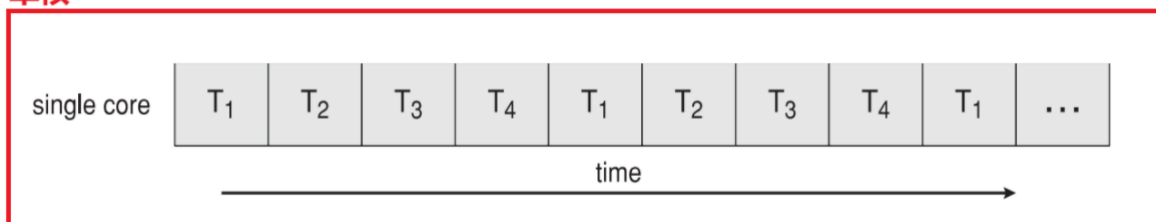
如果一个系统能够同时执行多个任务，则该系统是并行的。

- 如果一个系统能够同时执行多个任务，它就是并行的。
- 在多核系统中，每个核心可以独立执行任务，因此多个任务可以真正同时运行，而不是通过切换来实现。这就是 **真正的并行（parallelism）**。
- 例如，在一个四核系统中，四个任务可以分别在不同的核心上同时运行。

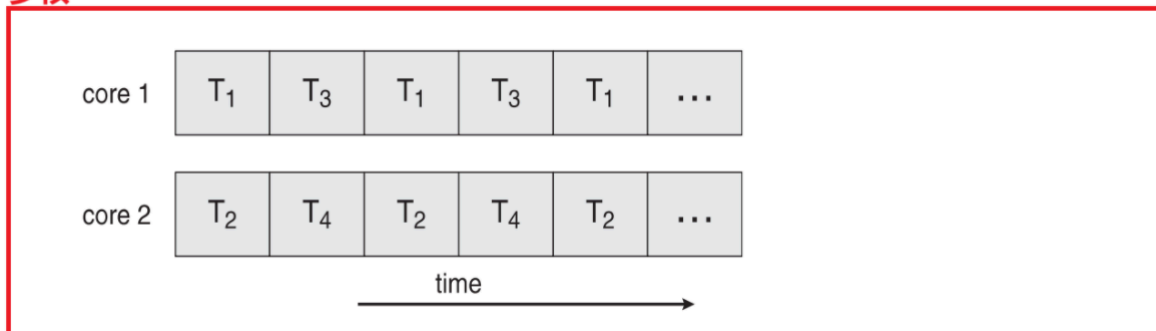
核心区别：

- **并发（Concurrency）**：任务**交替执行**，通过快速切换实现多个任务的“同时运行”。它更关注任务的调度和组织，适用于单核或多核系统。
- **并行（Parallelism）**：任务**真正同时执行**，通常需要多核或多处理器系统的硬件支持。

单核



多核



[3] 多线程模型

用户模式和内核模式

用户模式 (User Mode) 和内核模式 (Kernel Mode)

处理器会根据当前运行的代码类型在两种模式之间切换：

- 应用程序在 **用户模式** 下运行。

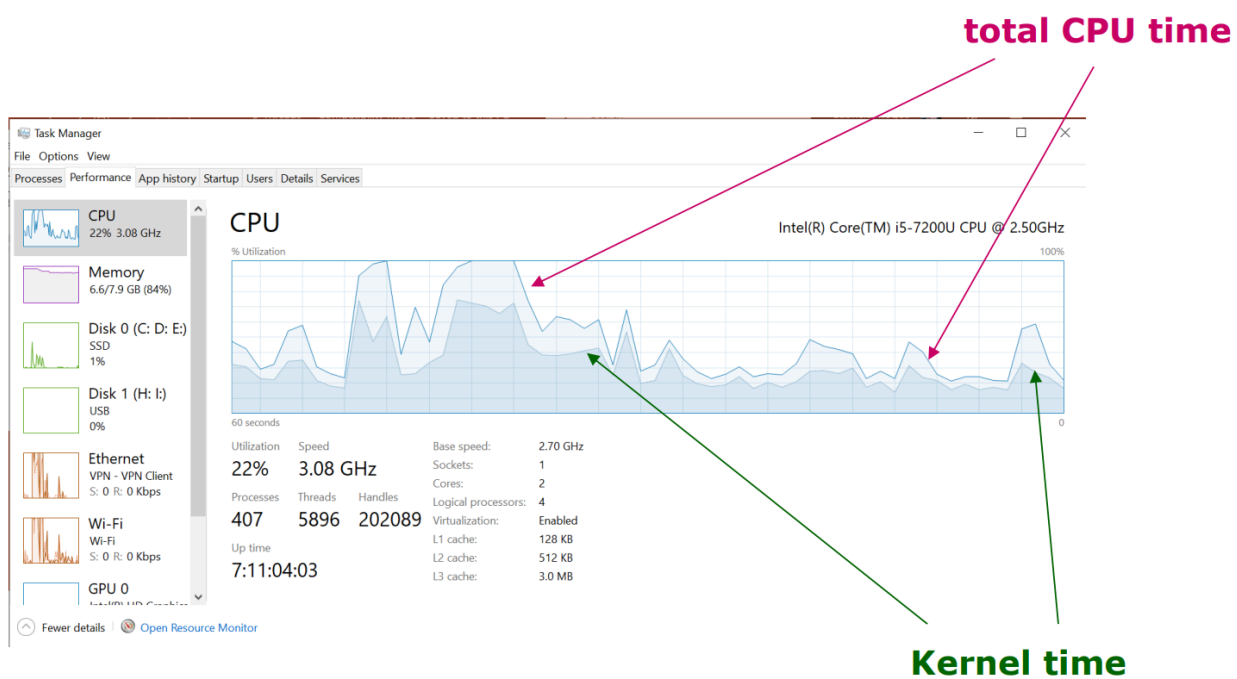
核心操作系统组件在内核模式下运行。

- 虽然许多驱动程序在内核模式下运行，但也有一些驱动程序可能在用户模式下运行。

使用任务管理器 **Ctrl** + **Alt** + **.** 查看 CPU 时间

其中最上面的一条线是总CPU时间，而下方深色部分是内核时间。那么两者之差（浅色部分）就是用户时间。

PPT 上说这是CPU时间，但实际上是利用率。



用户级线程和内核级线程

线程的类型有两种实现类别：

- 用户级线程 **User-Level Threads, ULT**
- 内核级线程 **Kernel-Level Threads, KLT**

用户级线程 ULT

用户级线程是由用户实现的执行单元，内核并不感知这些线程的存在。用户级线程的管理由**用户级线程库(ULT Library)**完成。

三个主要的线程库：

- POSIX Pthreads

- Windows 线程
- Java 线程
- 编程语言中的线程实现（如C#、Python中的线程）

内核级线程 KLT

内核级线程直接由操作系统处理，线程管理由内核完成。

几乎所有通用操作系统都采用内核级线程，包括：

- Windows
- Linux
- Mac OS X
- iOS
- Android

总结

- **用户级线程**：由用户空间实现，内核不可见，由 **用户级线程库** 管理。
- **内核级线程**：由操作系统内核直接管理，效率和调度更优。

用户级线程（ULT）与内核级线程（KLT）的区别

特性	用户级线程（ULT）	内核级线程（KLT）
创建和管理的速度	快	慢
实现方式	通过 用户级的线程库 实现。	由操作系统支持内核线程的创建。
通用性 Generality	用户级线程是通用的，可以运行在任何操作系统上。	内核级线程是 特定于 操作系统的。
多处理器利用	多线程应用程序无法利用多处理器（多核）的优势。	内核例程本身可以是多线程的，可以利用多处理器。
内核感知	内核并不感知用户线程的存在。	内核直接管理和调度内核线程。

[补充]

为什么 多线程应用程序无法利用多处理器（多核）的优势？

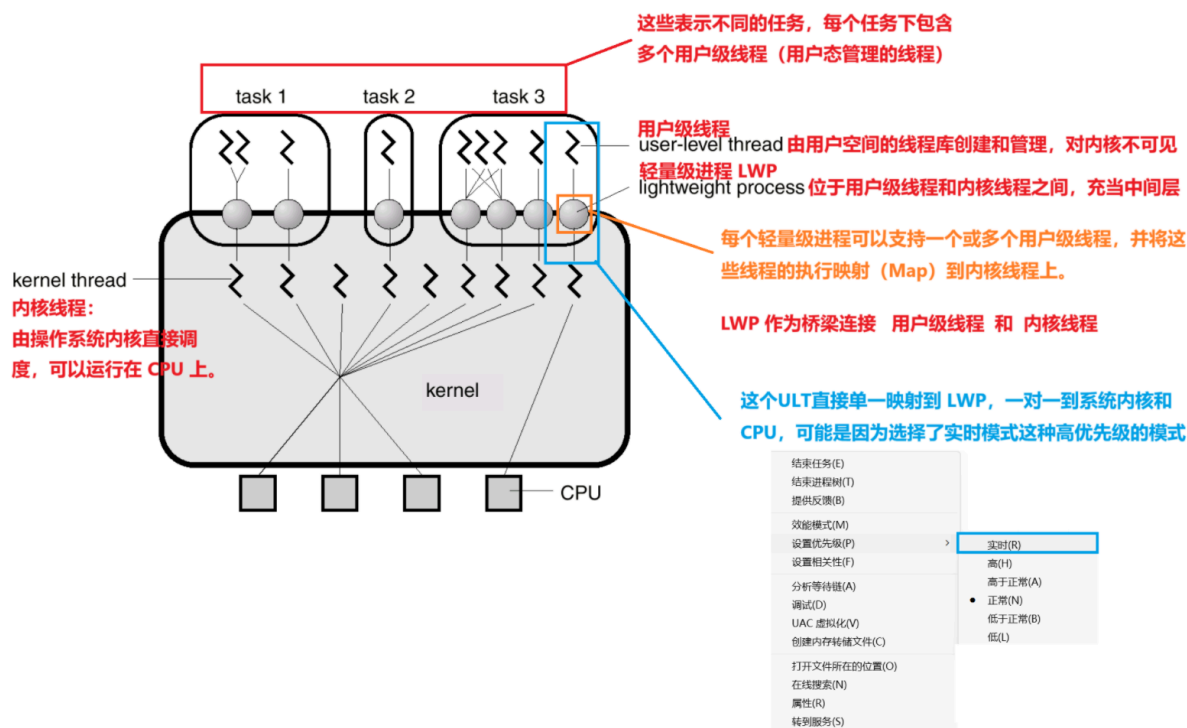
1. 用户级线程对内核不可见

2. 缺乏内核的调度支持

ULT 和 KLT 的关系

ULT: User-Level Thread 用户级线程

KLT: Kernel-Level Thread 内核级线程



上图展示了如何将**用户级线程 ULT** 映射为**内核级线程 KLT**

一个程序 (Task) 中的多个线程，可以在多个CPU处理器上并行运行。

多线程模型分为三种类型：

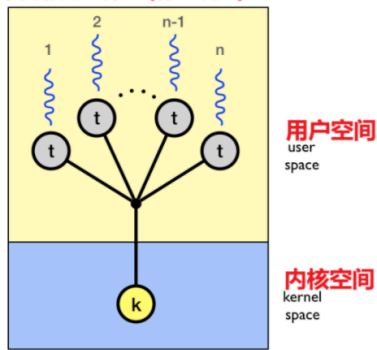
- 多对一 (Many-to-One) 多个 **ULT** 映射到单个 **KLT**
- 一对一 (One-to-One) 比如图上蓝色标注的 **ULT** 和 **KLT** 一对一
- 多对多 (Many-to-Many) 多个 **ULT** 映射到多个 **KLT**

多对一

多个 **ULT** 映射到单个 **KLT**

这种模型适用于：**不需要多处理器支持的简单应用**，但在 多核CPU的现代应用中可能存在局限。

多个用户级线程（标记为t）被映射到
单个内核级线程（标记为k）

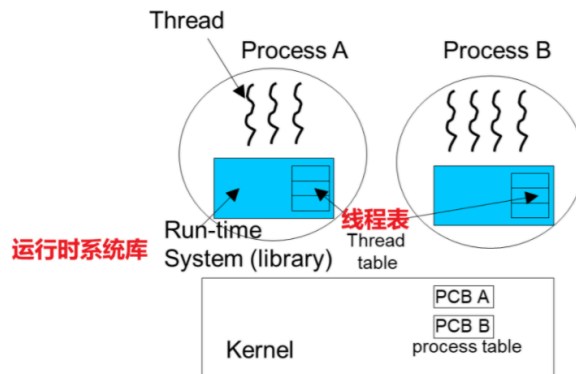


用户级线程运行在用户空间，由于是多对一模型，它们共享一个内核线程，因此在任意时刻只能有一个线程实际占用CPU。

内核空间中只有一个内核线程负责执行这些用户线程。

这种多对一模型的线程管理完全由用户级库负责

例如Solaris的Green Threads和GNU Portable Threads。



每个进程中都有一个运行时库，管理线程和线程表。该表维护所有用户线程的信息。

ULT 在进程内部由运行时系统进行切换，并不需要内核的直接参与。

内核只了解每个进程的进程控制块（PCB），而了解进程内部的线程。

关于上下文切换：

上下文切换（Context Switch）、并发执行，通常发生在内核中。

单核系统中的上下文切换：

- **线程上下文切换**：在一个单核系统中，CPU同一时刻只能执行一个线程。当操作系统决定暂停当前线程并开始执行另一个线程时，就会进行线程上下文切换。
- **进程上下文切换**：类似地，操作系统也可以在进程间进行切换。在这种情况下，需要保存和恢复更多的状态信息（例如，内存映射、文件描述符等），因为不同进程有各自独立的资源。

多核系统中的上下文切换：

- 在多核系统中，多个CPU核心可以同时独立运行多个进程或线程。
- **进程/线程上下文切换**：即使在多核系统中，一个核心上的进程或线程也可能需要被另一个进程或线程替换，这个过程也是上下文切换。
- 为什么还会发生上下文切换？尽管有多个核心，任务可能有更高的优先级需要被立即执行，或者当前任务需要等待I/O操作完成等。

图中右边的这种模型使线程管理变得灵活，但不支持在多处理器系统中的并行执行，因为内核只看到**单一的执行上下文**。（看完上下文切换的解释，应该就能理解这句话了）

一对一

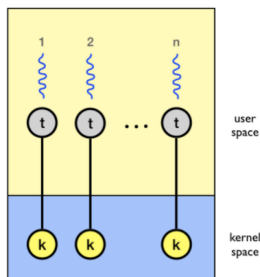
一对一模型

每个用户线程映射到一个内核线程

- 内核可以实现线程并管理线程、调度线程。
- 内核能够感知线程的存在。
- 提供了更多的并发性（Provides more concurrency）；当一个线程阻塞时，另一个线程可以继续运行。

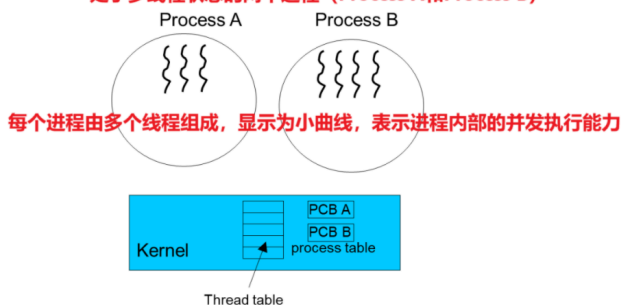
Examples

- Windows NT/XP/2000
- Linux
- Solaris 9 and later **处于多线程状态的两个进程 (Process A和Process B)**



每个ULT通过一条连接线映射到一个KLT

在这种模型中，每个ULT都有一个对应的KLT来为其提供 底层支持和调度



每个进程由多个线程组成，显示为小曲线，表示进程内部的并发执行能力

内核包含一个线程表和一个进程表。

线程表：管理所有线程的信息，包括与进程相关的信息。

进程表：包含进程控制块（PCB），用于管理进程的状态和资源。

在这种模型中，内核能够感知每个线程，因为每个用户线程都与一个内核线程直接映射。这意味着内核可以更有效地进行线程调度和资源管理。该模型的优点包括可以更好地支持并发，因为当一个线程阻塞时，另一个线程仍然可以运行。然而，代价是每个线程占用内核资源，可能导致较高的资源消耗。

多对多

允许将多个用户级线程映射到多个内核级线程

- 允许操作系统创建足够数量的 KLT
- KLT 的数量可以特定于某个特定应用程序或特定机器
- 用户可以创建任意数量的线程也就是ULT，并且相应的内核级线程可以在多处理器上**并行**运行

总结

并发和并行：

- 并发 (Concurrency)**：指的是系统能够在同一时间段内处理多个任务。并发意味着多个任务能够进展，但不一定同时执行。并发通常用于描述单核系统中不同任务通过时间切片机制交替执行的能力。
- 并行 (Parallelism)**：指的是在同一时刻真正同时执行多个任务。并行通常用于描述多核系统中，可以通过同时在多个核心上执行任务来提高计算效率。

单核系统中的并发

在单核系统中，程序通常通过**上下文切换**来实现并发。在这种环境下，多任务之间虽然不能并行执行，但可以通过快速切换任务来营造出同时进行的感覺。

多核系统中的并行

多核系统的设计使其可以真正同时执行多个任务

三种模型对比：

特性	一对一 (One-to-One)	多对一 (Many-to-One)	多对多 (Many-to-Many)
管理方式	内核管理	用户级线程库管理	用户级线程库和内核共同管理
线程映射机制	每个用户级线程对应一个内核线程	多个用户级线程映射到一个内核线程	多个用户级线程映射到多个内核线程
优点	1. 优化 并行性	1. 线程创建和切换速度快	1. 灵活性高，适应性强
	2. 线程阻塞不影响其他线程	2. 资源消耗小	2. 可以利用多核处理器
			3. 阻塞一个线程不影响其他线程
缺点	1. 创建和管理成本高	1. 不能利用多核处理器	1. 实现复杂度高
	2. 上下文切换开销大（因为一个 ULT 对应一个 KLT，切换的时候麻烦。）	2. 阻塞一个线程会阻塞所有线程	
适用场景	高并发和 并行 处理应用	轻量级线程且不需要并行处理的系统	复杂应用和异构工作负载

[4] 线程库

Thread Libraries

线程可以通过线程API（线程库的函数）来 创建、使用和终止。

API： Application Programming Interface 应用程序编程接口

线程库为程序员提供了一种**API**，用于创建和管理线程。

- 这个线程可以在用户空间或内核空间中实现。
- **用户级库 (User-level library)** （无需内核支持）。
- 库的所有代码和数据结构都**存在于用户空间中**。
 - 调用库中的函数会导致**用户空间中的本地函数调用**，而不是系统调用。
- **内核级库 (Kernel-level library)** （直接由操作系统支持）。
- 库的代码和数据结构存在于**内核空间中**。
 - 调用库API中的函数通常会导致对**内核的系统调用**。

三种主要的线程库：POSIX线程、Java线程、Win32线程。

[5] 隐式线程

Implicit threading

线程技术可以分为两类：**显式线程**和**隐式线程**。

- **显式(Explicit)线程**：程序员显式地创建和管理线程。
- **隐式(Implicit)线程**：**编译器**或**运行时库**负责创建和管理线程。

设计多线程程序的三种替代方案：

1. **线程池 Thread pool** - 在进程启动时创建一定数量的线程并将其放入池中，这些线程在池中等待任务。
2. **OpenMP** - 是一组适用于C、C++和Fortran程序的编译器指令，用于告诉编译器：在适当的地方自动生成**并行代码**。

一个简单的例子是，对于 `for` 循环，通过 OpenMP 优化，让不同的核心计算不同的区间段，加快 `for` 循环

```
1 // 使用 OpenMP 实现并行化
2 #pragma omp parallel for
3 for (i = 0; i < N; i++) {
4     b[i] = a[i] * a[i];
5 }
```

3. **Grand Central Dispatch (GCD)** - 是Apple的MacOS X和iOS操作系统上为C和C++提供的扩展，用于支持并行编程。

[5] 线程问题 / 设计多线程程序

Threading issues / Designing multithreaded programs

1. `fork()` 和 `exec()` 系统调用
2. 信号处理
3. 线程取消

`fork()` 和 `exec()` 系统调用

`fork()` 是 仅复制调用线程 还是所有线程？

`fork()` 会创建一个新进程，但在多线程程序中，它只会复制调用 `fork()` 的那个线程，而不会复制父进程中的所有线程。

因此，新创建的子进程将是单线程的。

`exec()` 系统调用

`exec()` 会用参数中指定的新程序替换当前进程的代码和数据，同时会终止当前进程的所有线程，并从新程序的入口点开始执行一个单线程的进程。

总结：

1. `fork()` 仅复制调用线程，子进程是单线程的。
2. `exec()` 替换当前程序及其所有线程，新程序以单线程运行。

信号处理

信号是一种软件中断，它发送给程序以指示某个重要事件已经发生。

信号在 **UNIX** 系统中被广泛使用。
Windows 系统没有显式支持信号。

所有信号的处理过程遵循以下模式：

1. 信号的**生成**：特定事件的发生会生成一个信号。
2. 信号的**传递**：该信号被传递给某个进程。
3. 信号的**处理**：信号一旦被传递，就必须被处理。

例子 1：

非法内存访问和除以零操作。

如果正在运行的程序执行了上述操作中的任何一个，就会生成一个信号。
信号会被传递给**引发该操作的同一进程**。

例子 2：

UNIX/Linux 系统

Ctrl-C

按下此键会导致系统向正在运行的进程发送一个 **INT** 信号（**SIGINT**）。默认情况下，此信号会立即终止（中断）该进程。

Ctrl-Z

按下此键会导致系统向正在运行的进程发送一个 **TSTP** 信号（**SIGTSTP**）。默认情况下，此信号会挂起该进程的执行。

Ctrl-\

按下此键会导致系统向正在运行的进程发送一个 **QUIT** 信号（**SIGQUIT**）。默认情况下，此信号会立即终止（杀死）该进程。

进程通过运行特殊的**信号处理程序**来捕获并处理信号。信号处理程序执行完毕后，OS 会从进程原先被中断的地方**重新启动该进程**。

信号可以由以下两种可能的处理程序来处理：

1. 默认信号处理程序

默认操作是指：应用程序没有捕获并处理信号时会发生的行为。

对于许多信号类型，默认操作是**立即终止进程**。在某些情况下，默认操作是**简单地忽略信号**。

2. 用户自定义信号处理程序

操作是用户定义的，可以根据需要自定义信号的处理方式。

线程取消

线程取消通常有两种主要方式：

- 1. **异步取消 (Asynchronous Cancellation)**
 - **立即终止**目标线程。
 - 如果要取消的线程正在更新共享数据，可能会导致问题。
- 2. **延迟取消 (Deferred Cancellation)**
 - 允许目标线程定期**检查是否应该被取消**。

与目标线程相关的关键问题包括：

- 如果已经为被取消的目标线程分配了资源，这些资源会如何处理？

[补充]

线程被取消时，资源可能未被释放，导致泄漏。

解决：使用清理函数（如 `pthread_cleanup_push`）或在线程取消点手动释放资源，确保资源被正确回收。

- 如果目标线程在更新与其他线程共享的数据时被终止，会发生什么情况？

[补充]

共享数据可能处于不一致状态，导致程序错误或死锁。

解决：使用锁机制保护共享数据，确保线程在终止前完成更新并释放锁，或使用原子操作保证数据一致性。

▲ [Week 2 Tutorial]

二进制乘除法

回顾上周讲了二进制数的加减法，先做一些练习回顾

练习回顾

(a) 将二进制数 1011 转换为十进制形式。

$$1 * 2^3 + 1 * 2^1 + 1 * 2^0 = 11$$

(b) 将二进制数 1.011 转换为十进制形式。

整数部分： $1 * 2^0 = 1$

小数部分： $0 * 2^{-1} + 1 * 2^{-2} + 1 * 2^{-3} = 0.25 + 0.125 = 0.375$

总：1.375

(c) 将数字 15 和 12 转换为二进制形式，将这两个二进制数相加，并将答案转换为十进制形式以验证和是否正确。

十进制转二进制：

15 的二进制是 1111_2 ，计算步骤如下

步骤	被除数（十进制）	商	余数
1	$15 \div 2$	7	1
2	$7 \div 2$	3	1
3	$3 \div 2$	1	1
4	$1 \div 2$	0	1

12 的二进制是 1100 ，计算步骤如下

步骤	被除数（十进制）	商	余数
1	$12 \div 2$	6	0
2	$6 \div 2$	3	0
3	$3 \div 2$	1	1
4	$1 \div 2$	0	1

因此

$$\begin{array}{r} 1\ 1\ 1\ 1 \\ +\ 1\ 1\ 0\ 0 \\ \hline 1\ 1\ 0\ 1\ 1 \end{array}$$

$$11011_2 = 2^4 + 2^3 + 2^1 + 2^0 = 16 + 8 + 2 + 1 = 27_{10}$$

(d) 将数字 9 和 6 转换为二进制形式。用此计算 $9 - 6$ 的二进制结果，并通过将二进制答案转换为十进制形式来验证其正确性。

同理， $9_{10} = 1001_2$ ， $6_{10} = 110_2$ 。

$$\begin{array}{r} 1\ 0\ 0\ 1 \\ -\ 1\ 1\ 0 \\ \hline 1\ 1 \end{array}$$

$$11_2 = 3_{10}$$

乘法

计算 7×5 ，首先转为二进制 $111_2 \times 101_2$

$$\begin{array}{r}
 \\
 \\
 \\
 \\
 \\
 \\
 \hline
 1
 \end{array}$$

得到 100011

除法

不带小数位的，例如 $15 \div 5 = 3 = 11_2$

$$\begin{array}{r}
 \\
 \\
 \\
 \\
 \\
 \\
 \hline
 1
 \end{array}$$

带小数位的，例如 $15 \div 6 = 2.5 = 10.1_2$

$$\begin{array}{r}
 \\
 \\
 \\
 \\
 \\
 \\
 \hline
 1
 \end{array}$$

■ [Week 3 Lecture]

进程同步 Process Synchronization

包含了Tutorial

进程同步

- ☐ Background of PS 进程同步的背景
- ☐ The Critical-Section Problem 临界区问题
- ☐ Peterson's Solution 彼得森算法
- ☐ Synchronization Hardware 同步硬件
- ☐ Mutex Locks / Mutual exclusion 互斥锁 / 互斥
- ☐ Semaphore 信号量
- ☐ Classical Problems of Synchronization 同步的经典问题

[1] 背景

什么是进程同步？

(Process Synchronization, PS)

进程同步是指：协调进程执行的任务，**确保在同一时间内，没有两个进程能够同时访问相同的共享数据和资源。**

多个进程（n个进程）竞争使用某些共享资源。

数据并发访问

对共享数据的**并发访问（Concurrent access）**可能导致数据不一致。

因此，维护数据一致性（Data consistency）需要机制来确保协作进程的有序执行。

竞争条件（Race condition）：指的是，多个进程**并发访问和操作共享数据**的情况。共享数据的最终值取决于哪个进程最后完成。

为了防止竞态条件，必须对并发进程进行同步(Synchronized)。

[2] 临界区问题

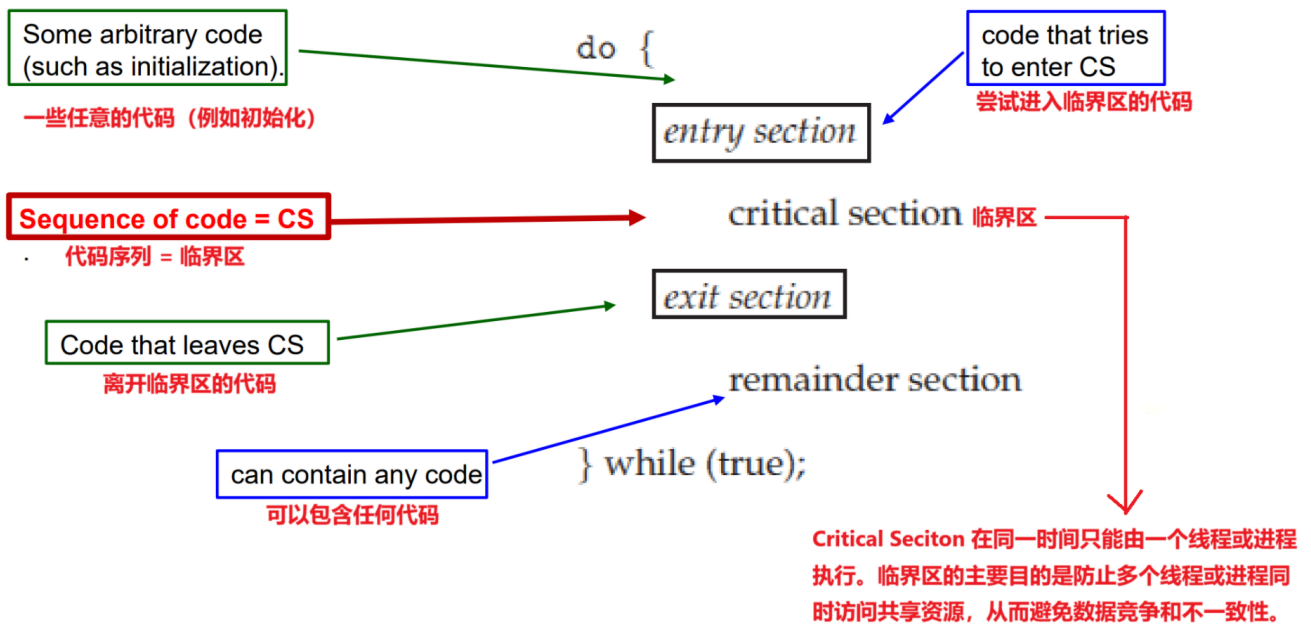
The Critical-Section Problem

什么是临界区？

临界区（Critical Sections）是指：**一段代码**，这段代码访问**共享资源（如共享数据或共享设备）**，并且在同一时间**只能由一个线程或进程执行**。

临界区的主要目的是防止多个线程或进程同时访问共享资源，从而避免数据竞争和不一致性。

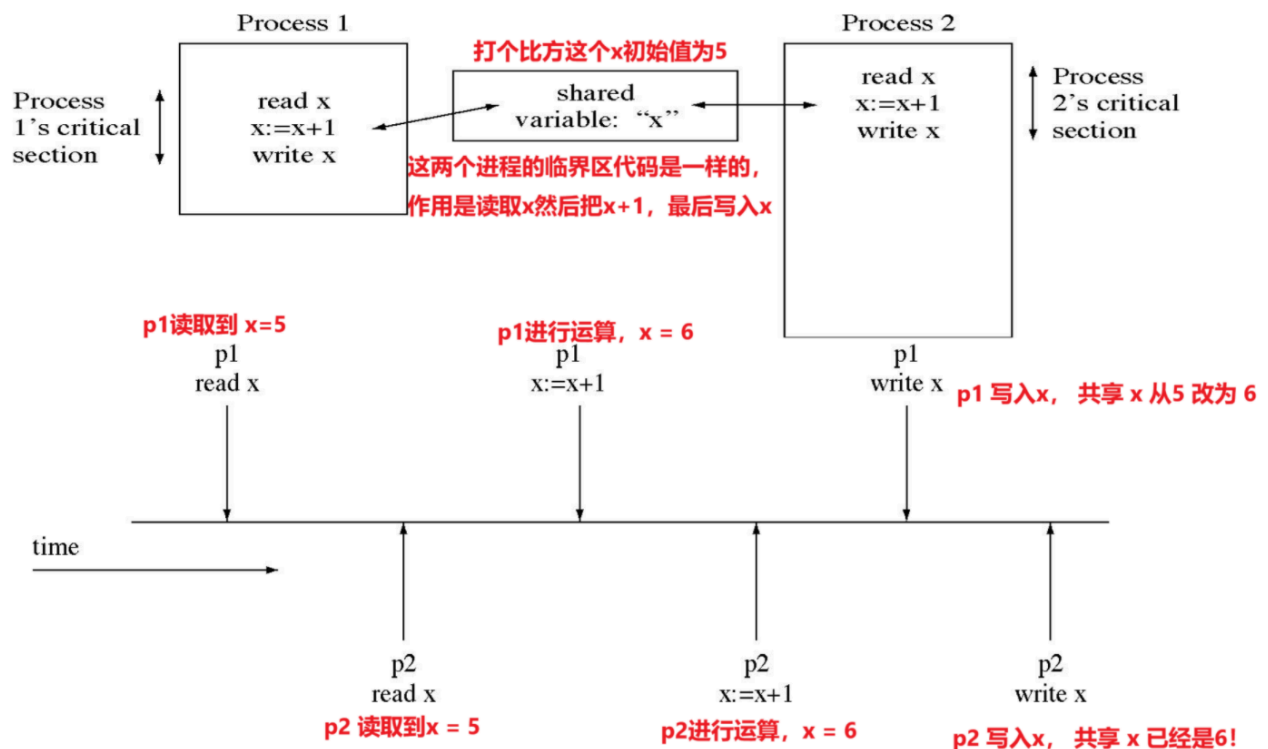
每个**并发的**线程/进程都有一个代码段，称为**临界区（Critical Section, CS）**，在使用临界区时，代码可以分为以下几个部分：



缺乏同步机制带来的临界区问题

Critical Section (CS, 临界区)：以“读-更新-写”方式引用一个变量的代码。

为什么同步机制非常重要？下面这张图是没有同步机制带来的数据不一致性。



原本预期的是 $x = 7$ 因为两个程序都执行了一次，但是结果却是6！原因很显然，是因为发生了竞争问题！

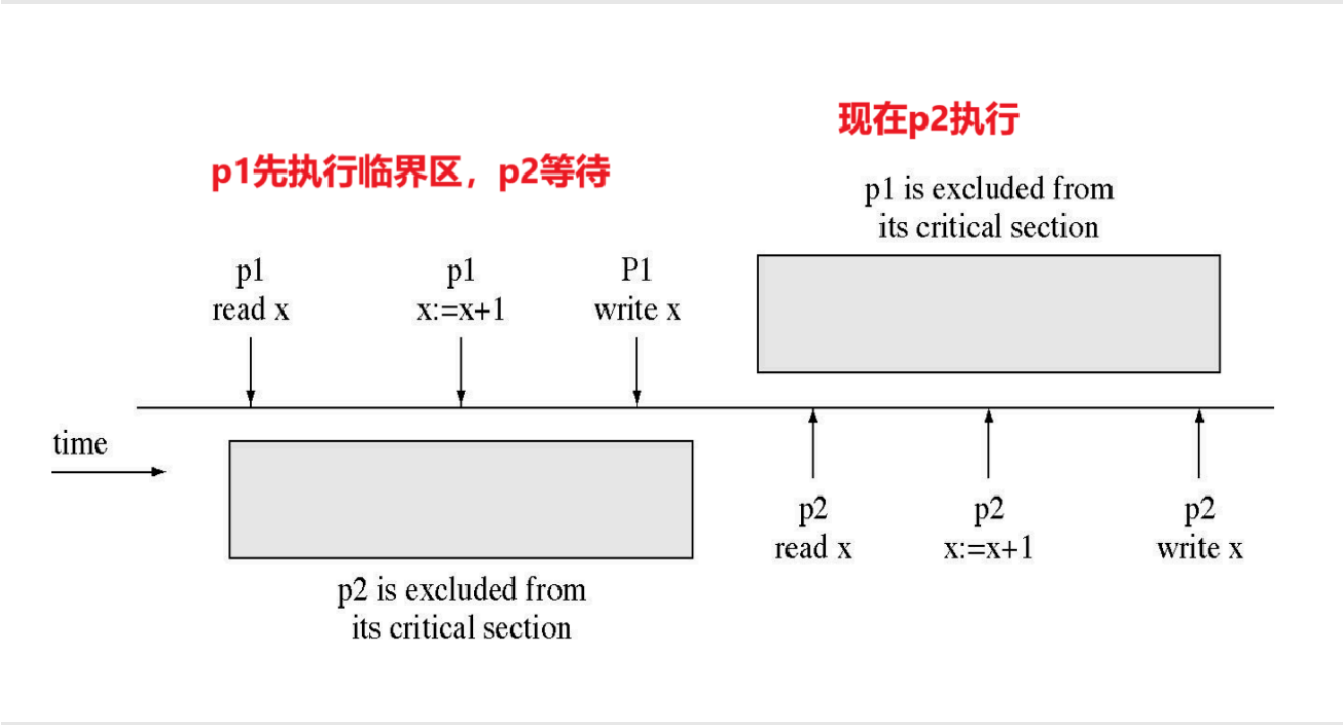
```
1  int i = 7; // 这个是我们需要更改的数量
2  void increment(){
3      i++;
4  }
5  int main(){
6      Thread T1, T2; // 创建两个线程 T1 和 T2
7      T1(increment()); // 让 T1 和 T2 都执行 increment
8      T2(increment());
9  }
```

那么运行结果会如何呢？

第一种可能，通过同步机制来确保数据正确性		第二种可能，发生了竞态条件（race condition）。导致了数据和结果不一致	
Thread 1	Thread 2	Thread 1	Thread 2
get i (7)	—	get i (7)	get i (7)
increment i (7->8)	—	increment i (7->8)	—
write back i (8)	—	—	increment i (7->8)
—	get i (8)	write back i (8)	—
—	increment i (8->9)	—	write back i (8)
—	write back i (9)		

同步机制解决临界区问题

那么如何避免上述情况呢？其实只需要 **避免Avoid/禁止Forbid/拒绝Deny** 在临界区内并行执行即可。



临界区解决方案

并发进程在使用相同资源（竞争性或共享性）时会发生冲突，例如：I/O设备、内存、处理器时间、时钟 (clock)。

要实现正确的解决方案，必须满足以下三个要求：

1. 互斥 (Mutual exclusion) '

互斥是指在同一时间内，**只能有一个进程访问临界资源**（即共享资源）。

这是为了避免多个进程同时修改共享资源，导致数据不一致或错误。

2. 进度 (Progress)

进度是指**系统能够推进进程的执行**，确保不会发生**死锁或饥饿**。具体来说：

- 如果一个进程不在临界区内，那么它**不能**阻止其他进程进入临界区。
- 系统应该能够选择下一个进入临界区的进程，而不是无限期地等待。

3. 有限等待 (Bounded waiting)

有限等待是指**每个进程在等待进入临界区时，等待的时间是有限的**。这是为了避免进程无限期地等待（即饥饿现象）。

- 饥饿**：某个进程因为资源一直被其他进程占用，导致它无法执行。
- 有限等待的实现**：通过公平的调度算法或优先级机制，确保每个进程最终都能获得资源。

示例：如果进程 A、B、C 都试图进入临界区，系统应确保进程 C 不会因为 A 和 B 的频繁访问而永远无法进入。

三者关系

- 互斥**是基础，确保资源的独占访问。
- 进度**确保系统能够推进进程的执行，避免死锁。
- 有限等待**确保每个进程都能在有限时间内获得资源，避免饥饿。

软件解决方案

通过算法：其正确性仅依赖于一个假设，即**同一时间只有一个进程/线程可以访问某个内存位置或资源**。

Peterson算法

Peterson算法**仅适用于两个进程，P0 和 P1**。该算法由Gary L. Peterson于1981年提出。

用于实现**互斥**，允许多个进程共享一个**单次使用的资源**而不会发生冲突，仅通过**共享内存**进行通信。

核心问题在于设计**进入区**和**退出区**：

```
1  do {
2      进入区
3          临界区 - CS
4      退出区
5      剩余区 - RS
6  } while(TRUE)
```

进程可以通过共享一些公共变量来同步它们的操作：

```
1  int turn; // 表示轮到哪个进程进入临界区
2  boolean flag[2]; // 初始化为 FALSE
3      // flag 表示某个进程是否想进入其临界区
4      // flag[i] = true 表示进程 Pi 已准备好 (i = 0,1)
```

需要同时使用 `turn` 和 `flag[2]` 来保证**互斥性**、**有限等待**和**进展性**。

实现如下：

```
1  // 程序 P0 如下
2  do {
3      // 现在进入临界区
4
5      flag[0] = TRUE; // 表示进程 P0 准备进入临界区
6
7      turn = 1;
8      while (flag[1] && turn == 1); // 这里什么事情都不干，一直循环
9
10     // 能运行到这里临界区就代表 flag[0] == true && turn = 1
11     CRITICAL SECTION // 临界区
12
13
14     flag[0] = FALSE; // 退出临界区，表示 P0 不再需要进入
15     REMAINDER SECTION // 剩余区
16 } while (TRUE); // 无限循环
```

```
1  // 程序 P1 如下
2  do {
3      // 现在进入临界区
4
5      flag[1] = TRUE; // 表示进程 P0 准备进入临界区
6
7      turn = 0;
8      while (flag[0] && turn == 0); // 这里什么事情都不干，一直循环
9
10     // 能运行到这里临界区就代表 flag[1] == true && turn = 0
11     CRITICAL SECTION // 临界区
12
13
14     flag[1] = FALSE; // 退出临界区，表示 P0 不再需要进入
15     REMAINDER SECTION // 剩余区
16 } while (TRUE); // 无限循环
```

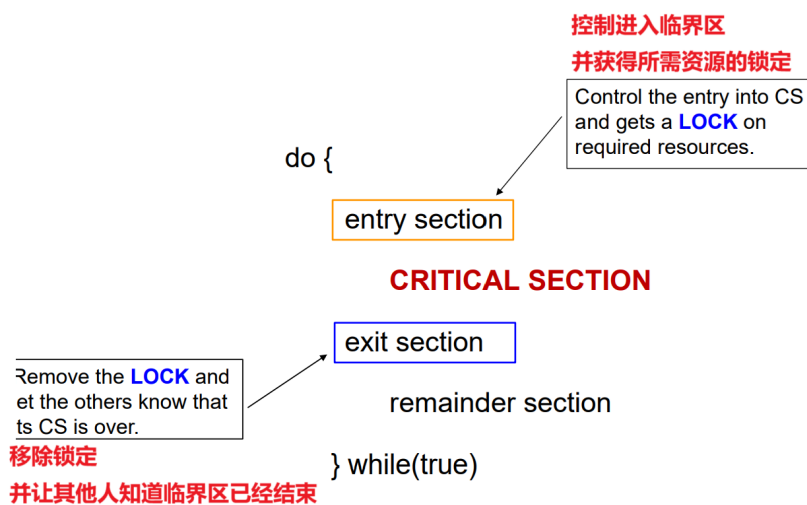
这个算法主要是用while 卡住执行。while的条件是 `&&`，其中如果turn是自己，但是flag另一个也成立，也就意味着：虽然自己要执行但是另外一个正在执行临界区，就会卡住，一直等。大概就是这个意思。

这个算法满足临界区解决方案的三个指标：

1. 互斥
2. 进度

硬件解决方案

依赖于特殊的机器指令来实现“锁定”



单处理器环境

通过禁用中断 (disable interrupt) , 可以有效地阻止操作系统调度其他进程运行。

单处理器环境: 指的是只有一个 CPU 的计算机系统, 同一时间只能运行一个进程。

TEST AND SET 解决方案

初始状态: `Lock` 值设置为 0

`Lock` = 0 表示临界区当前为空闲, 没有进程在其中。

`Lock` = 1 表示临界区当前被占用, 有一个进程在其中。

满足互斥要求, 但不幸的是, 不能保证有限等待。

多处理器环境下

提供了特殊的原子硬件指令。原子性意味着不可中断 (即指令作为一个单元执行) 。

- 一个全局变量 `lock` 被初始化为 0。
- 唯一能进入临界区 (CS) 的进程是发现 `lock = 0` 的进程。
- 这个进程通过将 `lock` 设置为 1 来排除所有其他进程。

关于原子性:

原子性意味着不可中断:

原子性是指一个操作在执行过程中不会被其他操作打断。在多处理器环境中, 即使有多个 CPU 同时运行, 原子指令也能保证在执行期间不会被其他 CPU 的指令干扰。

原子指令的执行是**不可分割的**，要么完整执行，要么完全不执行。它不会被分割成多个步骤，也不会会在执行过程中被其他操作插入。

举例说明

例如，`Compare and Swap (CAS)` 是一种常见的原子指令。它的操作包括：

- 读取某个内存位置的值。
- 比较该值是否等于预期值。
- 如果相等，则将该内存位置的值更新为新值。

由于 `CAS` 是原子的，即使多个 CPU 同时尝试执行 `CAS` 操作，硬件也会确保这些操作按顺序执行，不会导致竞争条件。

CAS(Compare and Swap) 解决方法

```
1  do { // 这是一个无限循环，表示进程或线程会不断尝试进入临界区。
2
3      while (compare_and_swap(&lock, 0, 1) != 0)
4          ; /* do nothing */
5
6      /* critical section */
7
8      lock = 0;
9
10     /* remainder section */
11 } while (true);
12
```

1. `do { ... } while (true);`

这是一个无限循环，表示进程或线程会不断尝试进入临界区。

2. `while (compare_and_swap(&lock, 0, 1) != 0)`

`compare_and_swap(&lock, 0, 1)` 是一个原子操作，它的作用是：

- 检查 `lock` 的值是否为 `0`（表示临界区未被占用）。
- 如果 `lock` 为 `0`，则将其设置为 `1`（表示占用临界区），并返回 `0`。
- 如果 `lock` 不为 `0`（表示临界区已被占用），则直接返回 `1`。一直执行循环。

3. 进入临界区

4. 执行完毕，恢复锁 `lock = 0`

5. 执行剩余区域

优缺点

优点

1. **适用于任意数量的进程**：无论是在单处理器系统上，还是在共享主存的多处理器系统上，都可以使用。
2. **简单且易于验证**：实现逻辑简单，容易理解和验证其正确性。
3. **支持多个临界区**：可以为每个临界区定义单独的变量，从而支持多个独立的临界区。

缺点

1. **忙等待 (Busy-waiting)**：当进程等待进入临界区时，它会持续占用处理器时间，导致 CPU 资源浪费。
2. **饥饿 (Starvation)**：如果一个进程长时间占用临界区，而其他多个进程也在等待进入，可能会导致某些进程长时间无法获得执行机会。
3. **死锁 (Deadlock)**：当一组进程相互等待某个事件（如临界区的释放），而这个事件只能由该组中的另一个被阻塞的进程触发时，可能会导致永久阻塞。

操作系统解决方案

提供特定的函数和数据结构，供程序员用于同步。

操作系统与编程语言解决方案

- 互斥锁 (Mutex)
- 信号量 (Semaphore)

互斥锁 (Mutex Lock) / 互斥 (Mutual Exclusion)

- 互斥锁是一种**编程标志**，用于获取和释放对象。
- 当开始处理那些无法同时执行的数据时，互斥锁会被设置为**锁定**状态，从而阻止其他程序对当前数据的操作。
- 当数据不再需要或任务完成时，互斥锁会被设置为解锁状态。

在内核级别强制互斥并防止共享数据结构损坏的最佳方法：在执行**最少数量**的指令时**禁用中断**。

内核级别的互斥：在内核中，多个进程或线程可能同时访问共享的数据结构（如队列、链表等）。如果没有互斥机制，可能会导致数据竞争（race condition），进而引发数据损坏或系统崩溃。

禁用中断的作用：禁用中断是一种**硬件级别的机制**，可以暂时阻止 CPU 响应外部中断（如时钟中断、I/O 中断等）。

- 当禁用中断时，当前执行的代码不会被其他进程或中断处理程序打断，从而确保当前操作是原子的（atomic）。
- 这种方法特别适合在内核中保护共享数据结构，因为它直接利用硬件特性，效率较高。

最小化禁用中断的时间：

- 禁用中断的时间应尽可能短，通常只覆盖需要保护的关键代码段（如修改共享数据结构的指令）。
- 如果禁用中断的时间过长，可能会导致系统无法及时响应外部事件，影响系统的实时性和稳定性。

这种方法通常用于内核中对性能要求极高的关键代码段，例如调度器、中断处理程序或锁的实现。

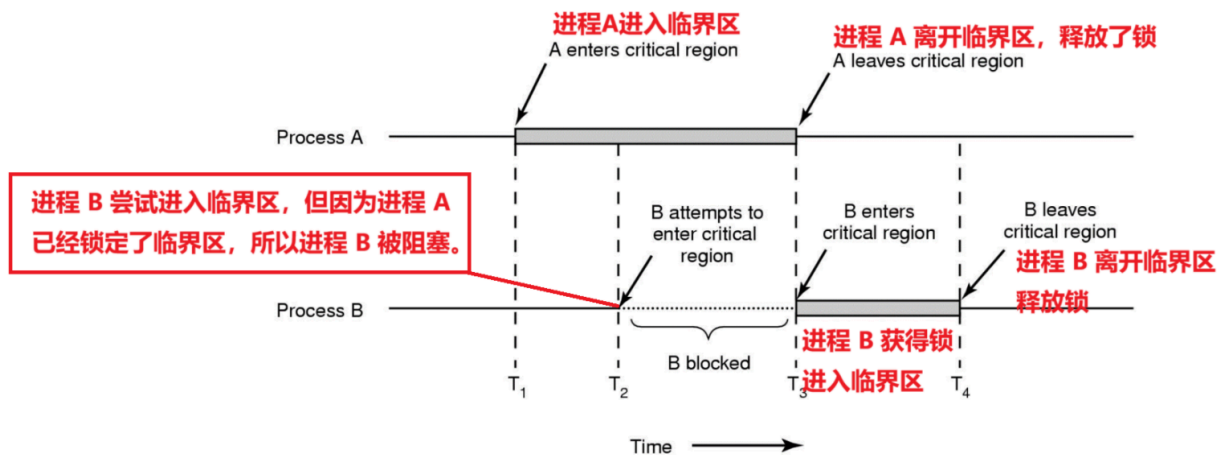
使用互斥锁的目的是在进入临界区之前获取锁，并在退出时释放锁。

```
do {  
    Acquire Lock  
  
    CRITICAL SECTION  
  
    Release Lock  
  
    REMAINDER SECTION  
  
} while (TRUE);
```

互斥对象由请求或释放资源的进程进行锁定或解锁。

在软件区域强制互斥的方法：使用忙等待机制。

- 忙等待机制是一种机制，其中进程会执行一个无限循环，等待锁变量的值指示其可用性。



这种类型的互斥锁被称为自旋锁(SpinLock)，因为进程在等待锁变得可用时会“自旋(spins)”等待。自旋锁的名称来源于等待锁的进程会“自旋”(spin)等待，即不断检查锁的状态，直到锁可用。这种等待方式通常在多核系统中使用，因为进程可以在等待锁时继续执行其他操作，而不是被阻塞。

图中显示了进程 B 在 T₂ 和 T₃ 之间被阻塞，等待进程 A 释放锁。这种等待方式在多核系统中可以提高效率，因为等待的进程可以在其他核心上执行其他任务。

信号量(Semaphore)

- 信号量(Semaphore)是由 Dijkstra 在1965年提出的。
- 它是一种通过使用一个简单的非负整数值来管理并发进程的技术，该值在多个线程/进程之间共享。
- 只能对信号量执行三种原子操作：初始化、递减和递增。
 - 递减操作可能导致进程被阻塞，
 - 递增操作可能导致进程被解除阻塞。
- 这个变量用于解决临界区问题，并在多处理环境中实现进程同步。

信号量 `S` 可以被初始化为一个非负整数值。

- 只能通过两种标准的原子操作访问：`wait()` 和 `signal()`。

- wait() 操作**递减**信号量值

如果 $S < 0$ ，则执行 `wait()` 的进程**被阻塞**。否则，执行自减。

```
1 wait(S){
2     while(S<0); // busy wait
3     S -- ;
4 }
```

- signal() 操作**递增**信号量值。

```
1 signal(S){
2     S++;
3 }
```

使用信号量解决临界区问题

- 对于 n 个进程
- 将信号量 S 初始化为 1
- 然后只允许**一个进程进入临界区**（互斥）
- 要允许 k 个进程同时进入临界区，只需将互斥量初始化为 k

进程 P_i :

```
1 do {
2     wait(S); // S初始化为 k，那么最先开始的k个程序不会卡在wait，而是直接通过wait把S减小到0
3             // 然后第k+1个程序就会卡在wait，一直等到前面k个程序的某个程序释放出 signal(S)
4     临界区
5     signal(S);
6     剩余部分
7 } while(true)
```

有两种主要类型的信号量：

- **计数信号量 (Counting Semaphore)**

允许任意资源计数。它的值可以在不受限制的域中变化。它用于控制对具有多个实例的资源的访问。 ($S = k$)

信号量 S 被初始化为可用资源的数量。

使用资源的每个进程都会在信号量上执行 `WAIT()` 操作（从而减少可用资源的数量）。

当进程释放资源时，它将执行 `SIGNAL()` 操作（增加可用资源的数量）。

当信号量的计数变为 0 时，所有资源都在使用中。之后，希望使用资源的进程将被阻塞，直到计数变为大于 0。

- **二进制信号量 (Binary Semaphore)**

类似于**互斥锁**。它只能有两个值：0 和 1。它的值被初始化为 1。它用于实现具有多个进程的临界区问题的解决方案。 ($S = 1$)

操作如上，不过是 $S = 1$

互斥锁 (Mutex Lock) 与 **二进制信号量 (BS)** 之间的主要区别在于：锁定互斥锁（将值设置为零）的进程必须是解锁互斥锁（将值设置为 1）的进程。

相反，一个进程可以锁定二进制信号量，而另一个进程可以解锁它（准确的意思是：）。（教程中的示例）

特性	互斥锁 (Mutex Lock)	二进制信号量 (Binary Semaphore)
所有权	锁定和解锁必须由同一个进程完成	锁定和解锁可以由不同的进程完成
主要用途	互斥访问共享资源	互斥访问或进程间同步
灵活性	较低，严格限制为同一进程操作	较高，允许多个进程协作

信号量的相同问题

- 1. **饥饿 (Starvation)** :
 - 当需要资源的进程被长时间延迟时，就会发生饥饿。
 - 高优先级的进程持续占用资源，导致低优先级的进程无法获取资源。
- 2. **死锁 (Deadlock)** :
 - 死锁是一种条件，其中没有进程能够继续执行，每个进程都在等待已被其他进程占用的资源。

[3] 同步的经典问题

- 1. **有限缓冲区问题 / 生产者-消费者问题 (The Bounded-Buffer / Producer-Consumer Problem)**
- 2. **读者-写者问题 (The Readers-Writers Problem)**
- 3. **哲学家进餐问题 (The Dining-Philosophers Problem)**

1. 有限缓冲区问题 / 生产者-消费者问题

有限缓冲区问题 / 生产者-消费者问题 是一个经典的计算机科学问题，它描述了一个生产者和消费者共享有限缓冲区的场景。这个问题通常用于说明同步和并发控制的重要性。

在这个问题中：

生产者 是一个进程或线程，负责生成数据并将其放入缓冲区。
消费者 是一个进程或线程，负责从缓冲区中取出数据并进行处理。

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```

有限缓冲区 是一个有固定容量的缓冲区，用于存储生产者生成的数据。生产者和消费者必须协调它们的活动，以确保缓冲区不会溢出（生产者不能在缓冲区满时继续插入数据）或空闲（消费者不能在缓冲区空时继续取出数据）。

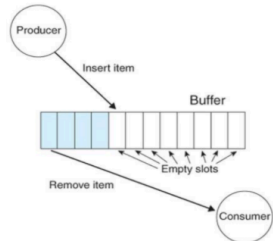
int n:: 表示缓冲区的大小，即缓冲区可以容纳的最大项目数。

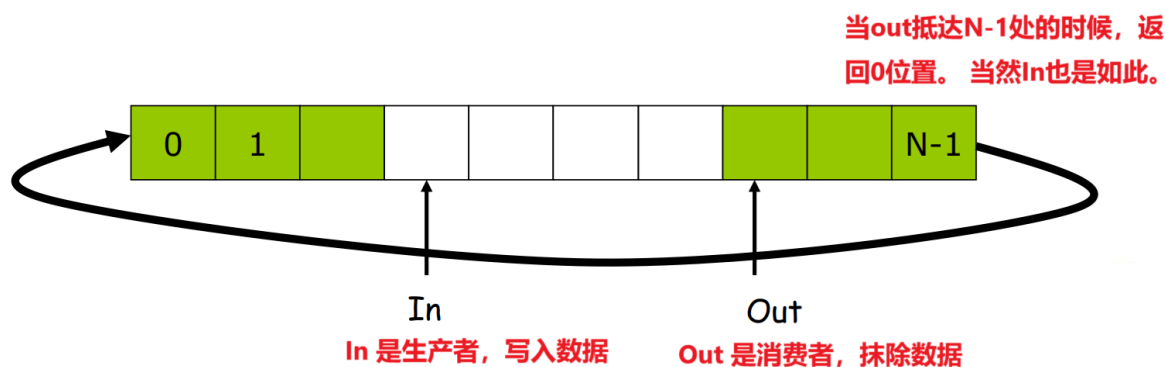
semaphore mutex = 1:: 互斥信号量 (mutex semaphore)，用于确保对缓冲区的互斥访问。它初始化为值 1。

semaphore empty = n:: 空信号量 (empty semaphore)，表示缓冲区中可用的空槽 (slots) 数量。它初始化为值 n，即缓冲区的大小。

semaphore full = 0:: 满信号量 (full semaphore)，表示缓冲区中已填充的槽 (slots) 数量。它初始化为值 0。

生产者 (Producer) 通过 Insert item 向缓冲区中插入项目。
消费者 (Consumer) 通过 Remove item 从缓冲区中移除项目。
空槽 (Empty slots) 表示缓冲区中当前可用的空槽。





- `mutex`，一个二进制信号量，用于获取和释放锁。
 - `empty`，一个计数信号量，其初始值是缓冲区中槽的数量，因为最初所有槽都是空的。
 - `full`，一个计数信号量，其初始值为 0。
- 在任何时刻，`empty` 的当前值表示缓冲区中空槽的数量，而 `full` 表示已占用槽的数量。

生产者大致函数如下

```

1  do
2  {
3
4      wait(empty); // 等待直到 empty > 0，然后递减 'empty'
5
6      wait(mutex); // 获取缓冲区的锁，以确保消费者在生产者完成操作之前无法访问缓冲区。
7                  // 等待锁=1
8                  // 获取到了mutex之后递减mutex
9
10     /* 在槽中执行插入操作 */
11
12     // 释放锁（递增mutex）
13     signal(mutex);
14     // 递增 'full'
15     signal(full);
16 }
17 while(TRUE)

```

消费者也是，大致如下

```

1  do
2  {
3      // 等待直到 full > 0，然后递减 'full'
4      wait(full);
5      // 获取锁，等待锁=1，递减mutex
6      wait(mutex);
7
8      /* 在槽中执行移除操作 */
9
10     // 释放锁，mutex++
11     signal(mutex);
12     // 递增 'empty'
13     signal(empty);

```

```
14 }
15 while(TRUE);
```

[来自Tutorial]

2. Reader - Writer 问题 经典同步问题

一组并发进程**共享**一个数据集。

- 多个读者可以同时读取资源，但写者必须独占资源（即写者运行时，其他读者或写者都不能访问资源）。

解决方案：

- 使用了三个变量：
 - **互斥锁** `m` 初始化为 1
 - 这是一个二进制信号量，用于保护 `read_count` 变量的更新。
 - `read_count` 是一个计数器，记录当前有多少读者正在访问资源。
 - 由于多个读者可能同时更新 `read_count`，所以需要用 `m` 来确保对 `read_count` 的更新是原子的。
 - **信号量 (Semaphore)** `w` 初始化为 1
 - 这是一个二进制信号量，用于确保写者独占资源。
 - 当写者运行时，`w` 会被占用，阻止其他写者和读者访问资源。
 - 当写者完成时，`w` 会被释放，允许其他进程访问资源。
 - **整数** `read_count` 初始化为 0，用于维护当前正在访问资源的读者数量。
 - 记录当前正在访问资源的读者数量。
 - 当 `read_count` 从 0 变为 1 时，第一个读者会占用 `w`，阻止写者访问资源。
 - 当 `read_count` 从 1 变为 0 时，最后一个读者会释放 `w`，允许写者访问资源。

写者进程的代码

```
1 while(TRUE)
2 {
3     wait(w); // 写者只需等待信号量 w，直到它为1，代表有机会向资源写入数据。然后 w--
4
5     /* 执行写操作 */
6
7     signal(w); // w++
8 }
```

读者进程的代码

```
1 while(TRUE)
2 {
3     // 第一部分：记录read_count，并且判断：如果当前存在读者，就不允许写者进行写入
4     wait(m); // m 用来保证 read_count 的更新防止race condition
5     read_count++; // 读者数量+1
```

```

6      if(read_count == 1) // 第一个读者出现了
7          wait(w);        // w 上锁，意思是不允许任何写者进行写入
8      signal(m);          // m 解锁，m--。
9
10     // 第二部分：读取Critical Section
11     /* 执行读操作 */
12
13
14     // 第三部分
15     wait(m); // m 用来保证 read_count 的更新防止race condition
16     read_count--;
17     if(read_count == 0)
18         signal(w); // 如果没有读者了，就进可以行写入，w++
19     signal(m);
20 }

```

3. 哲学家进餐问题

哲学家进餐问题是计算机科学中一个经典的同步问题，由荷兰计算机科学家 艾兹格·迪科斯彻（Edsger Dijkstra）提出，用于说明并发编程中资源竞争和死锁的问题。

Solution [1]: 1. 4个哲学家同时感到饥饿

原理：意思是只让1个人吃。能避免死锁，但是降低并发性（因为一次一个人吃）

Solution [2]: 只有当两双筷子都可用时才允许捡起筷子

避免了死锁，因为哲学家不会只拿一只筷子。缺点：降低并发性⁴



问题描述：

五个哲学家围坐在一张圆桌前，每个哲学家左右各有一双筷子。

桌上共有五双筷子，每双筷子位于两个哲学家之间。

哲学家们的行为分为两种：

思考：不需要筷子。（非临界区）

进餐：需要拿起左右两边的筷子才能进餐。（临界区）

2 目标是设计一种算法，确保哲学家们能够公平地进餐，同时避免以下问题：

死锁：所有哲学家同时拿起左边的筷子，导致没有人能拿到右边的筷子，从而无法进餐。

饥饿：某些哲学家永远无法拿到筷子，无法进餐。

Solution [3]: 奇数号哲学家总是先拿起左边的筷子，偶数号哲学家总是先拿起右边的筷子

原理：通过打破哲学家获取筷子的顺序，避免循环等待（死锁的核心原因）。

[Tutorial]

我们担心以下问题：

- **饥饿 (Starvation)：**一种策略可能会在某些情况下让某些哲学家一直处于饥饿状态（即使其他哲学家合作）。
- **死锁 (Deadlock)：**一种策略可能会导致所有哲学家都“卡住”，以至于没有人能做任何事情。
- **活锁 (Livelock)：**一种策略可能会让他们不停地做某件事，却永远无法吃到东西

[4] Quiz

1. 针对临界区问题，解决方案必须满足以下哪三个要求：

- A. 互斥 (Mutual exclusion)
- B. 进展 (Progress)
- C. 无界等待 (Un-Bounded waiting)
- D. 有界等待 (Bounded waiting)

答案：ABD

2. 在自旋锁 (Spinlocks) 中:

- A. 用于多处理器系统 (Employed on multiprocessor systems)
- B. 所有上述选项 (All of the mentioned)
- C. 锁的持有时间预期较短 (Locks are expected to be held for short times)
- D. 当进程需要等待锁时, 不需要上下文切换 (No context switch is required when a process must wait on a lock)

答案: B

解析:

自旋锁是一种忙等待 (Busy-waiting) 的锁机制, 常用于多处理器系统中。

自旋锁 Spinlocks 的工作原理:

- 1. **忙等待**: 当一个进程尝试获取自旋锁时, 如果锁已被其他进程持有, 该进程会进入一个循环, 不断检查锁的状态, 直到锁被释放。
- 2. **不放弃 CPU**: 在忙等待期间, 进程不会进入阻塞状态, **也不会让出 CPU**, 而是持续占用 CPU 资源。
- 3. **不执行其他任务**: 在忙等待期间, 进程不会执行其他任务, 而是专注于检查锁的状态。

适用场景:

自旋锁适用于以下场景:

- **锁持有时间较短**: 如果锁的持有时间很短, 忙等待的开销比上下文切换的开销更小。
- **多处理器系统**: 在多处理器系统中, 自旋锁的效率更高, 因为等待锁的进程可以在其他处理器上运行。

不适用场景:

- **单处理器系统**: 在单处理器系统中, 自旋锁会导致 CPU 资源浪费, 因为等待锁的进程会一直占用 CPU, 无法让其他进程运行。
- **锁持有时间较长**: 如果锁的持有时间较长, 忙等待会浪费大量 CPU 资源。

它的特点包括:

- 1. **用于多处理器系统**: 自旋锁适用于多处理器系统, 因为在单处理器系统中, 忙等待会浪费 CPU 资源。
- 2. **锁的持有时间预期较短**: 自旋锁适用于锁持有时间较短的场景, 因为长时间的忙等待会浪费 CPU 资源。
- 3. **不需要上下文切换**: 当进程需要等待锁时, 自旋锁不会让进程进入阻塞状态, 而是通过忙等待的方式不断检查锁的状态, 同时执行其他事情。因此不需要上下文切换。

3. 进程同步可以在__完成。

- A. **硬件和软件层面 (both hardware and software level)**
- B. **硬件层面 (hardware level)**
- C. **软件层面 (software level)**

D. 以上都不是 (none of the mentioned)

答案: A

进程同步 (Process Synchronization) 是操作系统中的一个重要概念, 用于协调多个进程或线程对共享资源的访问, 以避免竞争条件 (Race Condition) 和数据不一致的问题。进程同步可以通过以下方式实现:

1. 硬件层面:

- 硬件提供了一些原子操作 (如 Test-and-Set、Compare-and-Swap 等), 这些操作可以用于实现同步机制 (如自旋锁)。
- 硬件层面的同步通常效率较高, 但功能较为基础。

2. 软件层面:

- 操作系统提供了多种同步机制, 如信号量 (Semaphore)、互斥锁 (Mutex) 等。
- 软件层面的同步更加灵活, 功能更强大。

因此, 进程同步既可以在硬件层面实现, 也可以在软件层面实现。

4. 当多个进程并发访问同一数据, 且执行结果取决于访问发生的特定顺序时, 这种情况称为_____。

- A. 动态条件 (dynamic condition)
- B. 临界条件 (critical condition)
- C. 竞争条件 (race condition)
- D. 必要条件 (essential condition)

答案: C

竞争条件:

当多个进程或线程同时访问共享资源, 且最终结果依赖于它们的执行顺序时, 就会发生竞争条件。这种情况可能导致数据不一致或程序行为异常。

5. 如果一个进程正在执行其临界区, 那么其他进程不能执行它们的临界区。这种条件叫什么?

- A. 互斥 (mutual exclusion)
- B. 异步排除 (asynchronous exclusion)
- C. 同步排除 (synchronous exclusion)
- D. 临界排除 (critical exclusion)

答案: A

6. 系统中的哪个进程可能会受到其他进程执行的影响?

- A. init 进程 (init process)
- B. 父进程 (parent process)
- C. 协作进程 (cooperating process)
- D. 子进程 (child process)

答案: C

协作进程是指那些能够直接或间接地相互影响的进程。它们可能共享资源、数据或通过进程间通信 (IPC) 机制进行交互, 因此会受到其他进程执行的影响。

其他选项:

- **init 进程 (init process)**: 这是系统的第一个进程, 通常负责启动其他进程, 但它本身并不直接与其他进程协作。

- **父进程 (parent process)** : 父进程可能会创建子进程, 但并不一定与其他进程协作。
- **子进程 (child process)** : 子进程由父进程创建, 但并不一定与其他进程协作。

7. 以下哪个条件对于“进展 (Progress)”是正确的?

- A. 当一个线程正在执行其临界区时, 其他线程不能执行它们的临界区。
- B. 如果没有线程正在执行其临界区, 并且有一些线程希望进入它们的临界区, 那么其中一个线程将进入临界区。
- C. 多个进程访问和操作临界区中的相同数据。
- D. 以上所有。

答案: B

题目考察的是操作系统中关于“进展 (Progress)”的概念。进展是进程同步中的一个重要原则, 确保系统能够向前推进, 避免死锁或饥饿。

- **选项 A**: 描述的是**互斥 (Mutual Exclusion)**, 而不是进展。互斥确保同一时间只有一个线程可以执行临界区, 但这并不直接体现进展。
- **选项 B**: 正确描述了**进展 (Progress)**。如果没有线程正在执行临界区, 并且有线程希望进入临界区, 系统应该允许其中一个线程进入, 确保系统能够向前推进。
- **选项 C**: 描述的是**共享数据**的情况, 与进展无关。
- **选项 D**: 错误

8. 信号量是一个共享的整型变量 __

- A. **不能大于零 (that can not be more than zero)**
- B. **不能低于一 (that can not drop below one)**
- C. **不能大于一 (that can not be more than one)**
- D. **不能低于零 (that can not drop below zero)**

答案: D

选项 A: 错误。信号量的值可以大于零, 表示可用资源的数量。

选项 B: 错误。信号量的值可以低于一, 为零, 表示没有可用资源。

选项 C: 错误。信号量的值可以大于一, 具体取决于可用资源的数量。

选项 D: 正确。信号量的值**不能低于零**, 因为负值没有实际意义, 且信号量的值始终是非负整数。

9. 选择关于使用互斥锁 (mutex lock) 防止竞态条件(race condition)的正确陈述。

- A. 进程在进入临界区之前必须获取锁;
- B. 进程在进入临界区之前不需要获取锁;
- C. 进程在退出临界区时释放锁;
- D. 进程在退出临界区时必须获取锁。

答案: AC

- 陈述 A 正确: 进程在进入临界区之前必须获取锁, 以确保互斥访问。陈述 B 错误
- 陈述 C 正确: 进程在退出临界区时释放锁, 以允许其他进程进入临界区。
- 陈述 D 错误。

10. **Bounded Waiting** 是什么意思

- A. 当一个线程在其临界区执行时, 其他线程不能在其临界区执行。

- B. 如果没有线程在其临界区执行，并且有一些线程希望进入其临界区，那么其中一个线程将进入临界区。
- C. 多个进程并发地访问和操作相同的数据。
- D. 在一个线程请求进入其临界区后，其他线程被允许进入其临界区的次数是有限的，直到该请求被批准。

答案：D。After a thread makes a request to enter its critical section, there is a bound on the number of times that other threads are allowed to enter their critical sections, before the request is granted

A：这是**互斥**（Mutual Exclusion）的定义，而不是**Bounded Waiting**。（When a thread is executing in its critical section, no other threads can be executing in their critical sections）

B：这是**进展**（Progress）的定义，而不是**Bounded Waiting**。（If no thread is executing in its critical section, and if there are some threads that wish to enter their critical sections, then one of these threads will get into the critical section）

C：这是**并发访问**的描述，与**Bounded Waiting**无关。（Several processes access and manipulate the same data concurrently）

11. 互斥可以通过__提供。

- A. both mutex locks and binary semaphores（互斥锁和二进制信号量）
- B. none of the mentioned（以上都不是）
- C. binary semaphores（二进制信号量）
- D. mutex locks（互斥锁）

答案：A

12. 信号量 S 是一个整型变量，除了初始化之外，只能通过两个标准的原子操作来访问：

答案：wait() 和 signal()

13. 每个进程都有一段代码，称为 __，在这段代码中，进程可能会修改公共变量、更新表、写入文件等。

答案：临界区（Critical Section）

14. Peterson 的解决方案仅限于 __ 个进程，这些进程在它们的临界区和剩余区之间交替执行。

答案：2

Peterson 的解决方案是一种经典的进程同步算法，用于解决两个进程之间的互斥问题。它通过使用两个共享变量（turn 和 flag）来确保同一时间只有一个进程进入临界区。由于该算法的设计仅适用于两个进程，因此它无法扩展到多个进程的场景。

所以，填空的正确答案是 2。

● Lab

基于 Lab。部分非PPT会标注，作为补充。

○ [Lab1]

在学期的第一部分，将快速介绍 C 语言的基础知识。在学期的第二部分，我们将学习 Linux 操作系统的基础知识以及 C 语言的开发。

规范：

```
1  #include <stdio.h>                // 总是使用这个库！
2  int main() {
3      printf("Hello world!");
4      return 0;                    // 总是 return 0;为结尾
5  }
```

转义字符

尝试输出

```
1  Hello world!
2  I'm learning "C in Linux" coding
```

```
1  #include <stdio.h>
2  int main(){
3      printf("Hello world!\n");
4      printf("I'm learning \"C in Linux coding\"\n");
5      return 0;
6  }
```

C++中一些需要转义的字符

转义字符	含义
<code>\n</code>	换行符 (Newline)
<code>\t</code>	水平制表符 (Tab)
<code>\b</code>	退格符 (Backspace)
<code>\r</code>	回车符 (Carriage Return)
<code>\f</code>	换页符 (Form Feed)
<code>\a</code>	响铃符 (Alert/Bell)
<code>\\</code>	反斜杠 (Backslash)
<code>\'</code>	单引号 (Single Quote)
<code>\"</code>	双引号 (Double Quote)

循环

```
1 Test case 1 :
2
3 Input:
4 5
5
6 Output:
7 0x5 = 0
8 1x5 = 5
9 2x5 = 10
10 3x5 = 15
11 4x5 = 20
12 5x5 = 25
13 6x5 = 30
14 7x5 = 35
15 8x5 = 40
16 9x5 = 45
17 10x5 = 50
```

```
1 #include <stdio.h>
2 int main(){
3     int number;
4     scanf("%d",&number);
5     for(int iterate = 0; iterate <=10 ;iterate++){
6         printf("%dx%d = %d\n",iterate,number,number*iterate);
7     }
8     return 0;
9 }
```

基本数据类型和输入输出

非PPT内从

1. 整型 (Integer Types)

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
short	短整型	2 字节	-32,768 到 32,767	0 到 65,535	scanf("%hd", &a);	printf("%hd", a);
int	整型	4 字节	-2,147,483,648 到 2,147,483,647	0 到 4,294,967,295	scanf("%d", &a);	printf("%d", a);
long	长整型	4 或 8 字节	取决于平台	取决于平台	scanf("%ld", &a);	printf("%ld", a);
long long	超长整型	8 字节	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	0 到 18,446,744,073,709,551,615	scanf("%lld", &a);	printf("%lld", a);
unsigned short	无符号短整型	2 字节	0 到 65,535		scanf("%hu", &a);	printf("%hu", a);

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
<code>unsigned int</code>	无符号整型	4 字节	0 到 4,294,967,295		<code>scanf("%u", &a);</code>	<code>printf("%u", a);</code>
<code>unsigned long</code>	无符号长整型	4 或 8 字节	0 到取决于平台		<code>scanf("%lu", &a);</code>	<code>printf("%lu", a);</code>
<code>unsigned long long</code>	无符号超长整型	8 字节	0 到 18,446,744,073,709,551,615		<code>scanf("%llu", &a);</code>	<code>printf("%llu", a);</code>

2. 浮点型 (Floating-Point Types)

数据类型	描述	大小 (通常)	精度 (十进制有效位数)	范围 (大致)	输入格式	输出格式
<code>float</code>	单精度浮点数	4 字节	6-7 位	$\pm 3.4e-38$ 到 $\pm 3.4e38$	<code>scanf("%f", &a);</code>	<code>printf("%f", a);</code>
<code>double</code>	双精度浮点数	8 字节	15-16 位	$\pm 1.7e-308$ 到 $\pm 1.7e308$	<code>scanf("%lf", &a);</code>	<code>printf("%lf", a);</code>
<code>long double</code>	扩展精度浮点数	10 或 16 字节	19-20 位	取决于平台	<code>scanf("%Lf", &a);</code>	<code>printf("%Lf", a);</code>

3. 字符型 (Character Types)

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
<code>char</code>	字符型	1 字节	-128 到 127	0 到 255	<code>scanf("%c", &a);</code>	<code>printf("%c", a);</code>
<code>unsigned char</code>	无符号字符型	1 字节	0 到 255		<code>scanf("%c", &a);</code>	<code>printf("%c", a);</code>
<code>wchar_t</code>	宽字符型 (用于 Unicode 字符)	2 或 4 字节	取决于平台		使用 <code>wscanf</code> 和 <code>wprintf</code> 特殊处理	

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
<code>char16_t</code>	16 位 Unicode 字符型 (C++11 引入)	2 字节	0 到 65,535		使用 <code>wscanf</code> 和 <code>wprintf</code> 特殊处理	
<code>char32_t</code>	32 位 Unicode 字符型 (C++11 引入)	4 字节	0 到 4,294,967,295		使用 <code>wscanf</code> 和 <code>wprintf</code> 特殊处理	

4. 布尔型 (Boolean Type)

数据类型	描述	大小 (通常)	范围	输入格式	输出格式
<code>bool</code>	布尔型 (真或假)	1 字节	<code>true</code> 或 <code>false</code>	使用 <code>int</code> 类型间接输入	使用 <code>int</code> 类型间接输出

5. Void 类型

数据类型	描述	输入格式	输出格式
<code>void</code>	表示“无类型”，通常用于函数返回值或指针。	无输入	无输出

关于 Boolean

与 Java 不同，C 语言中没有专门的 `boolean` 类型，而是使用 `int`、`double` 和 `char` 中的特殊值来表示 `false`：

- 对于 `int`，`0` 表示 `false`。
- 对于 `double`，`0.0` 表示 `false`。
- 对于 `char`，空字符 `'\0'` 表示 `false`。

在 C 语言中，以下值被视为 `FALSE`：`0`、`0.0`、`'\0'`。
除此之外，其他值都被视为 `TRUE`。

!1 是 0：

- `1` 在逻辑运算中通常表示 **真**。
- 逻辑非 (NOT) 操作会将 **真** 反转为 **假**，所以 `!1` 的结果是 `0` (表示 **假**)。

!0 是 1：

- `0` 在逻辑运算中通常表示 **假**。
- 逻辑非 (NOT) 操作会将 **假** 反转为 **真**，所以 `!0` 的结果是 `1` (表示 **真**)。

数组

在C语言中如果你没有初始化一个数组，那么它里面的值就是随机的。

练习

Ex 1

你计划今晚入住的旅馆提供非常有趣的价格，前提是你不要到得太晚。客房服务在中午前完成房间的准备工作，客人越早到达中午之后，需要支付的费用就越少。请编写一个 C 程序，根据你的到达时间计算你需要支付的价格。

你的程序将读取一个整数，表示你到达时间距离中午的小时数。例如，0 表示中午到达，1 表示下午 1 点到达。基础价格为 10 元，每超过中午一小时增加 5 元。幸运的是，总价格上限为 53 元，因此你永远不需要支付超过这个金额。你的程序应根据输入的到达时间打印出你需要支付的价格（一个整数）。

测试用例

测试用例编号	输入	输出
1	7	45
2	10	53

```
1  #include<stdio.h>
2  int main(){
3  int price = 10; // basic price
4  int arrivalTime = 0;
5  scanf("%d", &arrivalTime);
6  while(arrivalTime-- > 0){
7  price += 5;
8  }
9  if(price >53)
10 price = 53;
11 printf("%d\n", price);
12 }
```

Ex 2

你的祖父母给了你一个非常棒的饼干食谱。食谱中有10种成分，每种成分所需的量（以克为单位）作为输入给出。

你的程序需要读取10个整数（按顺序表示每种成分所需的量），并将它们存储在一个数组中。然后，程序读取一个整数，表示成分的编号（1到10之间），并输出对应的量。

测试用例

测试用例1:

- 输入:

```
1 | 200 180 650 15 600 36 420 1 370 2
2 | 5
```

- 输出:

```
1 | 600
```

```
1 | #include <stdio.h>
2 | int main() {
3 |     int component[10];
4 |     for(int i = 0; i < 10; i++) {
5 |         scanf("%d", &component[i]);
6 |     }
7 |     int index;
8 |     scanf("%d", &index);
9 |     printf("%d\n", component[index-1]);
10 | }
```

Ex 3

你想为拔河锦标赛的决赛下注。在比赛前，选手的名字和体重会交替展示（先展示队伍1的选手1，然后是队伍2的选手1，接着是队伍1的选手2，依此类推）。每支队伍的选手数量相同。你需要记录选手的体重并计算总重量，以便为你的下注提供依据。

你的程序首先需要读取一个整数，表示每支队伍的成员数量。然后，程序需要交替读取每支队伍的选手体重。

程序接下来需要显示哪支队伍具有优势，即总重量更大的队伍。然后，显示每支队伍的总重量，如测试用例1所示。

输入:

```
1 | 4
2 | 110
3 | 106
4 | 113
5 | 102
6 | 112
7 | 121
8 | 117
9 | 111
```

输出:

```
1 Team 1 has an advantage
2 Total weight for team 1: 452
3 Total weight for team 2: 440
```

```
1 #include <stdio.h>
2 int main() {
3     int numberOfEachTeam = 0;
4     scanf("%d", &numberOfEachTeam);
5     int totalPowerOfTeamA = 0;
6     int totalPowerOfTeamB = 0;
7     for (int i = 0; i < numberOfEachTeam; i++) {
8         int temp;
9         scanf("%d", &temp);
10        totalPowerOfTeamA += temp;
11        scanf("%d", &temp);
12        totalPowerOfTeamB += temp;
13    }
14    if (totalPowerOfTeamA > totalPowerOfTeamB)
15        printf("Team 1 has an advantage\n");
16    else
17        printf("Team 2 has an advantage\n");
18    printf("Total weight for team 1: %d\n", totalPowerOfTeamA);
19    printf("Total weight for team 2: %d\n", totalPowerOfTeamB);
20 }
```

Ex 3

你负责管理一列由多个车厢组成的货运列车。你发现有些车厢超载，对铁轨造成了过大的压力，而有些车厢则过轻，存在安全隐患。于是你决定重新分配重量，使得所有车厢的重量完全相同，同时不改变总重量。你编写了一个程序来帮助你完成重量的分配。

你的程序首先需要读取车厢的数量（整数），然后读取每个车厢的重量（双精度浮点数）。接着，程序需要计算并显示每个车厢需要增加或减少的重量，使得所有车厢的重量相同。所有车厢的总重量不应改变。这些增加或减少的重量应显示为保留一位小数的浮点数。

你可以假设车厢数量不超过20个。

输入：

```
1 5
2 40.0
3 12.0
4 20.0
5 5.0
6 33.0
```

输出：


```
1 | -18.0
2 | 10.0
3 | 2.0
4 | 17.0
5 | -11.0
```

○ [Lab2]

While 循环

Ex1.

给定若干整数输入，代表每个月份的营业额，请你累加他们。读取到 `-1` 为止。

Input:

```
1 | 2500 1000 1500 -1
```

Output:

```
1 | 5000
```

```
1 | #include<stdio.h>
2 | int main(){
3 |     int sum = 0; // lab运行的话这个要初始化为0
4 |     int scannedNumber = 0;
5 |     scanf("%d",&scannedNumber);
6 |     while(scannedNumber!=-1){
7 |         sum+=scannedNumber;
8 |         scanf("%d",&scannedNumber);
9 |     }
10 |    printf("%d\n",sum);
11 | }
```

String and Char

在 C++ 中，定义 `char[]`（字符数组）的方式如下：

1. 定义一个固定大小的字符数组

```
1 | char name[20]; // 定义一个可以容纳 19 个字符（最后一个字符为 '\0'）的数组
```

这里 `name` 是一个可以存储最多 19 个字符的数组，最后一个字符是 `'\0'`（空字符），用于表示字符串的结束。

2. 初始化字符数组

在定义的同时可以初始化字符数组：

```
1 | char name[] = "Hello"; // 编译器会自动计算数组大小（包括 '\0'）
```

这里 `name` 的大小会被自动计算为 6（包括字符串的结束符 `'\0'`）。

3. 逐个字符初始化

也可以逐个字符初始化：

```
1 | char name[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

4. 注意事项

- **数组大小**：如果手动指定数组大小，必须确保数组足够大以存储所有字符和 `'\0'`。
- **字符串长度**：如果初始化时使用字符串字面量（如 `"Hello"`），数组大小会自动包括 `'\0'`。
- **访问元素**：可以通过索引访问数组中的字符，例如 `name[0]` 是 `'H'`。

5. 更多char 数组操作

计算char数组长度

```
1 | #include <cstring> // 包含 strlen 的头文件
2 | char str[] = "Hello";
3 | int length = strlen(str); // length = 5
```

复制char数组

```
1 | #include <cstring>
2 | char src[] = "Source";
3 | char dest[10];
4 | strcpy(dest, src); // 将 src 复制到 dest
```

char数组链接

```
1 | #include <cstring>
2 | char str1[20] = "Hello";
3 | char str2[] = " world!";
4 | strcat(str1, str2); // 将 str2 连接到 str1
```

截取部分字符长度

```

1 #include <cstring>
2 char src[] = "Hello, world!";
3 char dest[10];
4 strncpy(dest, src, 5); // 截取前 5 个字符
5 dest[5] = '\0';       // 手动添加结束符
6 cout << dest;         // 输出: Hello

```

清空这个数组

```

1 char str[] = "Hello";
2 str[0] = '\0'; // 清空字符串
3 cout << str;   // 输出为空

```

Ex2.

Write a program to read pairs of first and last names and display them in the order of last name followed by first name

编写一个程序来读取姓名的姓氏和名字，并按姓氏后跟名字的顺序显示它们

Assume that each first and last name has at most 10 characters and does not contain any spaces

假设每个名字（包括姓氏和名字）的长度最多为 10 个字符，且不包含任何空格

Your program should first read the total number of names (an integer)

您的程序应首先读取名称总数（一个整数）

Next, your program should read a first name and last name and then display these names is one line, the last name followed by one space, followed by the first name

接下来，您的程序应读取一个姓氏和一个名字，然后在一行中显示这些名字，姓氏后跟一个空格，然后是名字

Test case 1 :

Input:

```

1 4
2 Alan Turing
3 Ada Lovelace
4 Donald Knuth
5 Claude Shannon

```

Output:

```

1 Turing Alan
2 Lovelace Ada
3 Knuth Donald
4 Shannon Claude

```

```

1  #include <stdio.h>
2  int main() {
3      char firstName[50];
4      char lastName[50];
5      int totalNumberOfPeople;
6      scanf("%d", &totalNumberOfPeople);
7      while (totalNumberOfPeople --) {
8          scanf("%s %s", firstName, lastName);
9          printf("%s %s\n", lastName, firstName);
10     }
11 }

```

递归和函数自定义

Ex 3.

完成将公制单位转换为英制单位测量的骨架代码。测量值以米、克或摄氏度为单位提供给您的程序，必须分别转换为英尺、磅和华氏度，其中：

1 米 = 3.2808 英尺； 1 克 = 0.002205 磅； 华氏度 = 32 + 1.8 × 摄氏度。

在第一行输入中，您将获得需要进行转换的数量。随后的每一行包含要转换的值以及其单位：m、g 或 c（分别代表米、克或摄氏度）。

数值和单位之间会有一个空格。以 2 位小数的形式显示转换后的值，后跟一个空格和其单位：ft、lbs 或 f（分别代表英尺、磅或华氏度）。

Ex 4.

斐波那契数列

```

1  #include <stdio.h>
2
3  // write the prototype below
4  int fibo(int,int);
5  int sumUp(int n);
6
7  // complete the main function to read input, call functions, and display output
8  int main() {
9      int number;
10     scanf("%d",&number);
11     printf("%d\n",fibo(number,0));
12     return 0;
13 }
14
15 // complete the function below
16 int fibo(int iterativeTime, int sum) {
17     if(iterativeTime == 0)
18         return 1;
19     if (iterativeTime == 1)
20         return 1;
21     sum = fibo(iterativeTime -1, sum) + fibo(iterativeTime-2, sum);
22     return sum;

```